

কাজ: ভাষা বদলের ব্যবস্থা। সব লেখা অনুবাদ-ফাইল থেকে আসবে, কোডে হার্ডকোড হবে না।

#	বাটন / অংশ	API	ব্যাকএন্ডে কী ঘটবে (টেবিল)
1	**বাংলা বাটন**	UI `local: lang=bn`	ফোনে ভাষা সংরক্ষণ; লগইনের আগে সার্ভার কল লাগে না।
2	**English বাটন**	UI `local: lang=en`	সব লেবেল i18n ফাইল (bn.json / en.json) থেকে আসবে।
3	**শুরু করুন**	UI → `M02` লগইন	লগইন সফল হলে ভাষা সার্ভারে যায়: settings.language।



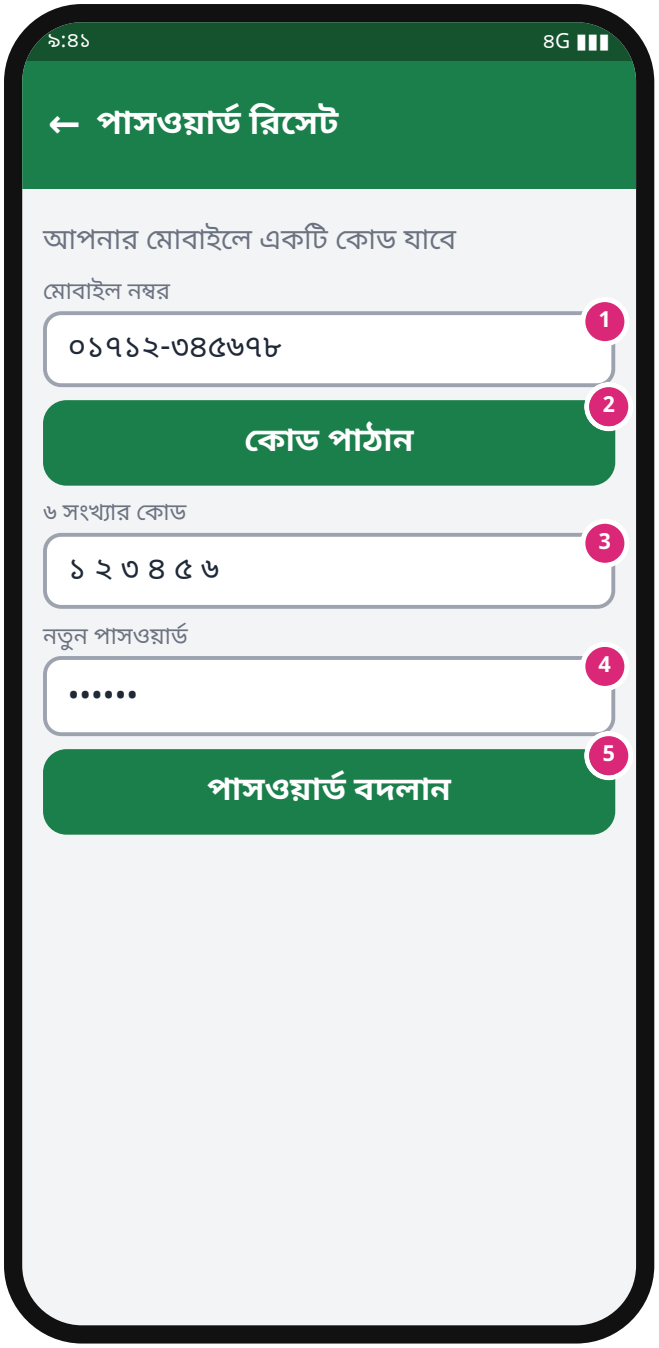
কাজ: লগইন ও সেশন। প্রতিটি পরের API-তে ব্যবসার আইডি টোকেন থেকে আসে, ক্লায়েন্টের পাঠানো ID বিশ্বাস করা হয় না।

#	বাটন / অংশ	API	ব্যাকএন্ডে কী ঘটবে (টেবিল)
1	**মোবাইল / ইমেইল ঘর**	UI –	ক্লায়েন্টে ফরম্যাট চেক; সার্ভারেও শেয়ার্ড ভ্যালিডেশন (zod)।
2	**পাসওয়ার্ড ঘর**	UI –	চোখ বাটনে দেখা/লুকানো। প্লেইন টেক্সট কোথাও সেভ হবে না।
3	**ঢুকুন**	POST `/auth/login`	users থেকে খোঁজা → bcrypt/argon2 মিলানো → সেশন/টোকেন তৈরি → business_members থেকে ব্যবসা ও রোল। ভুল হলে বাংলা বার্তা। ৫ বার ভুলে অস্থায়ী লক।
4	**পাসওয়ার্ড ভুলে গেছেন**	UI → M04	
5	**নতুন ব্যবসা খুলুন**	UI → M03	
6	**ডেমো দেখুন**	POST `/auth/demo-login`	আলাদা DEMO BUSINESS-এ ঢোকায়; আসল ব্যবসার সাথে কখনো মেশে না।



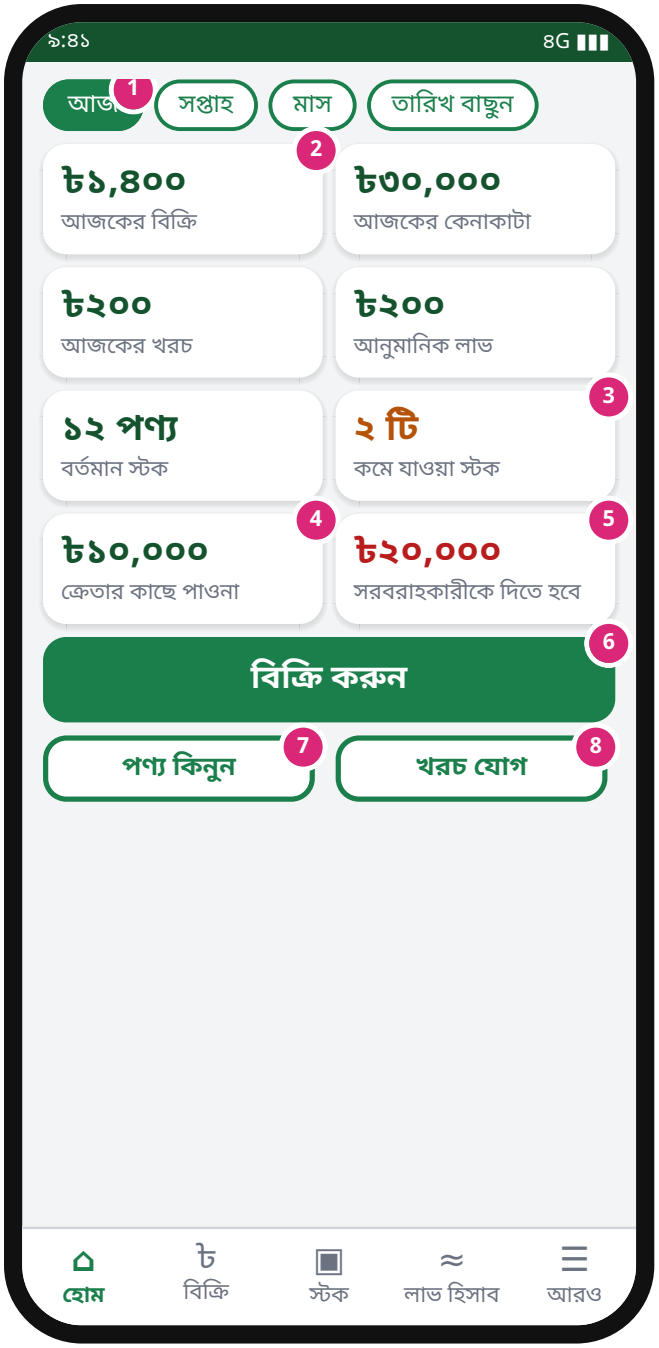
ব্যবহারকারী শুধু ৪টি তথ্য দেয়; বাকি সব ডিফল্ট সেট হয়।

#	বাটন / অংশ	API	ব্যাকএন্ডে কী ঘটবে (টেবিল)
1	**নাম**	UI –	users.name
2	**মোবাইল**	UI –	users.phone (ইউনিক); OTP যাচাই পরের ধাপে করা যাবে।
3	**ব্যবসার নাম**	UI –	businesses.name
4	**ব্যবসার ধরন**	UI –	businesses.type (ডেমো পণ্য-ধরন বাছাইয়ে কাজে লাগে)
5	**পাসওয়ার্ড**	UI –	ন্যূনতম ৬ অক্ষর; হ্যাশ হয়ে password_hash-এ।
6	**ব্যবসা খুলুন**	POST `/auth/register`	এক ট্রানজ্যাকশনে: users + businesses + business_members(role=Owner) + roles(ডিফল্ট Owner/ Staff) + categories(ডিফল্ট) + settings(ভাষা=bn, মুদ্রা=BDT) + audit_logs।



নিরাপত্তা: কোড একবার ব্যবহারযোগ্য; সীমিত চেষ্টা।

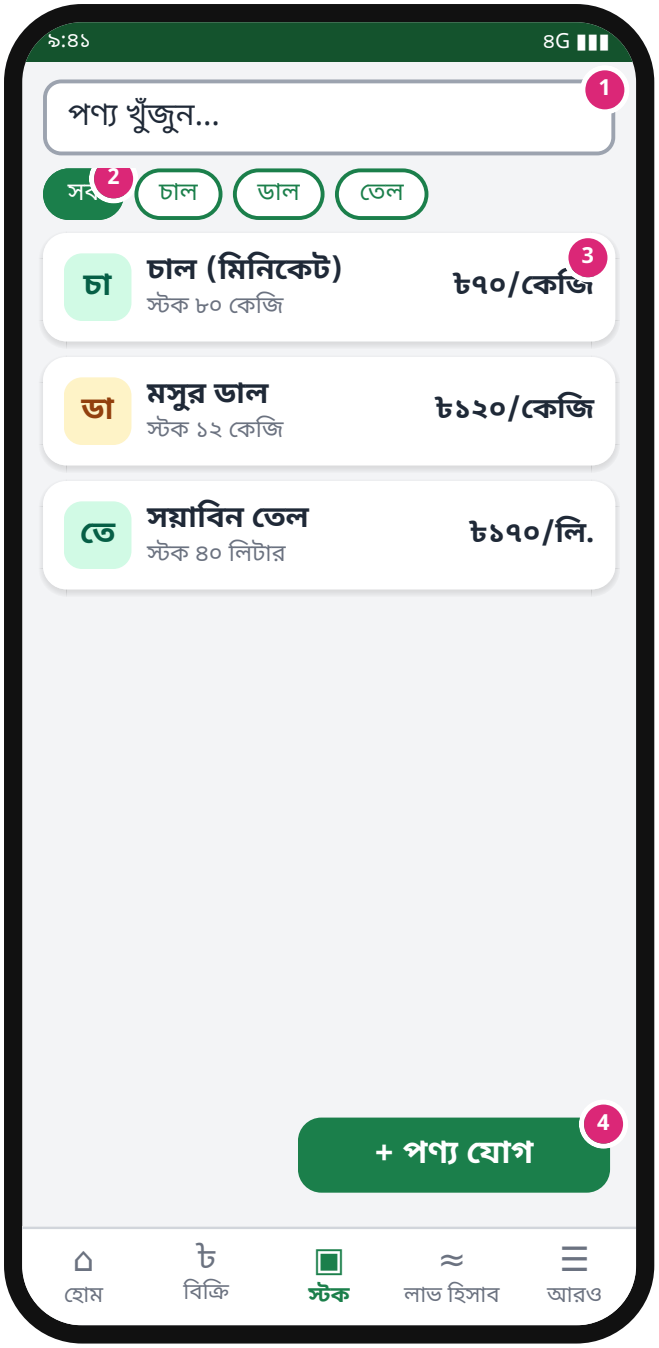
#	বাটন / অংশ	API	ব্যাকএন্ডে কী ঘটবে (টেবিল)
1	**মোবাইল নম্বর**	UI –	
2	**কোড পাঠান**	POST `/auth/password-reset/request`	password_resets-এ হ্যাশ করা কোড + ১০ মিনিট মেয়াদ। কোড পাঠানোর প্রোভাইডার (SMS/ইমেইল) আলাদা সার্ভিস; MVP-তে ডেভ মোডে লগে দেখাবে।
3	**কোড ঘর**	UI –	
4	**নতুন পাসওয়ার্ড**	UI –	
5	**বদলান**	POST `/auth/password-reset/confirm`	কোড মিললে password_hash আপডেট, পুরনো সব সেশন বাতিল, audit_logs।



আনুমানিক লাভ = আজকের বিক্রির মোট লাভ − আজকের খরচ। “আনুমানিক” লেখা স্পষ্ট থাকবে।

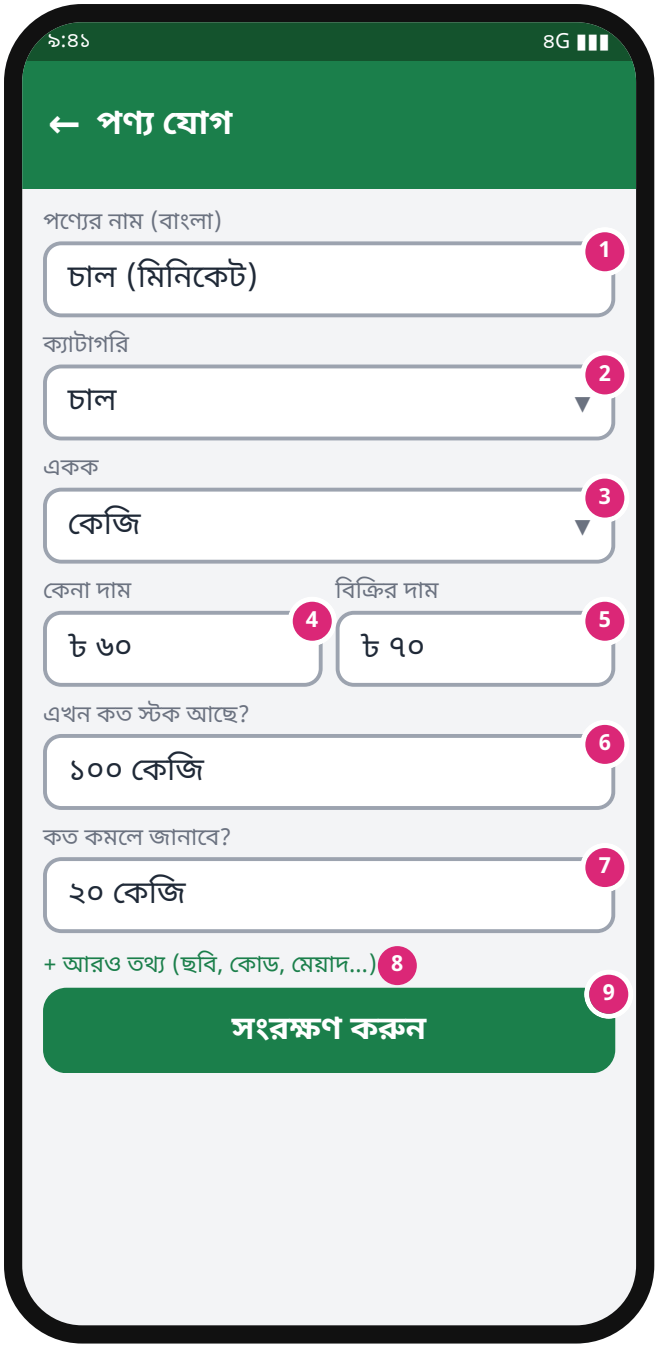
#	বাটন / অংশ	API	ব্যাকএন্ডে কী ঘটবে (টেবিল)
1	**সময়ের ফিল্টার (আজ/সপ্তাহ/মাস/নিজে বাছা)**	GET `/reports/dashboard?range=today|week|month|custom&from&to`	সব কার্ড একসাথে এক API-তে। হিসাব হয় sales, purchases, expenses, inventory থেকে সরাসরি; কোনো সংখ্যা হার্ডকোড নয়।
2	**আজকের বিক্রি কার্ড**	GET (ওপরের API)	SUM(sales.total) — শুধু active (বাতিল নয়) বিক্রি। এটি “বিক্রির টাকা”, লাভ নয়।
3	**কমে যাওয়া স্টক**	GET `/inventory/low-stock`	products.current_stock ≤ min_stock; ট্যাপ করলে M20।
4	**ক্রেতার কাছে পাওনা**	GET `/payments/receivables/summary`	SUM(customers.balance>0); ট্যাপ করলে M33।
5	**সরবরাহকারীকে দিতে হবে**	GET `/payments/payables/summary`	SUM(suppliers.balance>0)।
6	**বিক্রি করুন**	UI → M09	—
7	**পণ্য কিনুন**	UI → M15	—
8	**খরচ যোগ করুন**	UI → M21	—

আরও দুটি দ্রুত বাটন (লাভ হিসাব করুন → M23, Invoice তৈরি করুন → M34) স্ক্রল করলে নিচে দেখা যাবে; হোমে মোট ৬টি বড় বাটন থাকবে।



কম স্টকের পণ্য কমলা রঙে। ফাঁকা হলে M43-এর খালি স্ক্রিন।

#	বাটন / অংশ	API	ব্যাকএন্ডে কী ঘটবে (টেবিল)
1	**খোঁজার ঘর**	GET `/products?search=`	নাম(বাংলা/ইংরেজি)/কোড দিয়ে ILIKE খোঁজা; টাইপ থামলে ০.৪ সেকেন্ড পরে কল।
2	**ক্যাটাগরি চিপ**	GET `/products?category_id=`	categories-এর তালিকা /categories থেকে।
3	**পণ্য কার্ড**	UI → M08	ট্যাপ করলে বিস্তারিত।
4	**+ পণ্য যোগ**	UI → M07	



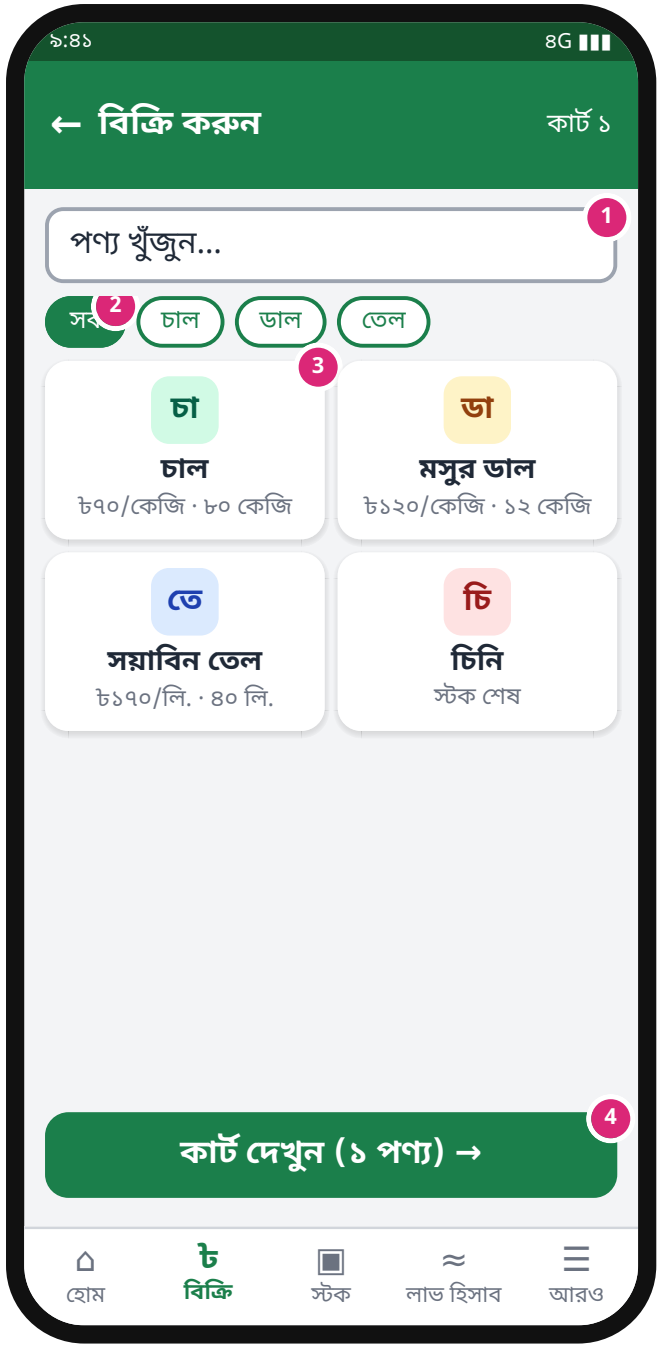
শুধু ৭টি ঘর বাধ্যতামূলক; ছবি ও অন্য ঘর ঐচ্ছিক।

#	বাটন / অংশ	API	ব্যাকএন্ডে কী ঘটবে (টেবিল)
1	**নাম**	UI –	products.name_bn (ইংরেজি নাম ও কোড ঐচ্ছিক)
2	**ক্যাটাগরি**	GET POST `/categories`	না পেলে নতুন ক্যাটাগরি যোগ।
3	**একক**	GET `/units`	কেজি, গ্রাম, লিটার, পিস, প্যাকেট, বক্স, ব্যাগ, ডজন + কাস্টম।
4	**কেনা দাম**	UI –	products.purchase_price NUMERIC(14,2)
5	**বিক্রির দাম**	UI –	products.selling_price। কেনার চেয়ে কম হলে সতর্কবার্তা (আটকায় না)।
6	**শুরুর স্টক**	UI –	সরাসরি স্টক বসে না; “শুরুর স্টক” নামে inventory_movements-এ এন্ট্রি হয়।
7	**সর্বনিম্ন স্টক**	UI –	products.min_stock
8	**আরও তথ্য**	UI –	ঐচ্ছিক: ছবি, SKU, মেয়াদ, ব্যাচ, গ্রেড, পছন্দের সরবরাহকারী।
9	**সংরক্ষণ**	POST `/products`	এক ট্রানজ্যাকশনে: products + inventory_movements(type=opening) + audit_logs। একই নাম+একক থাকলে বাধা।



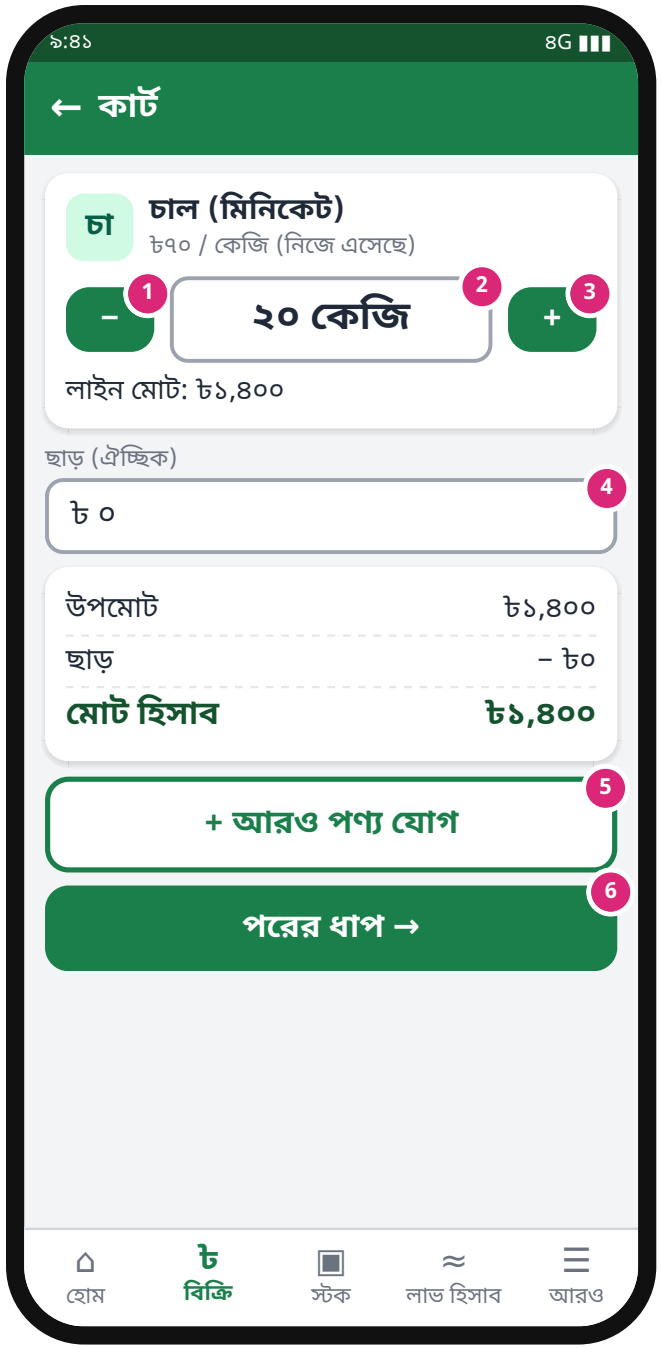
শুধু Owner দাম বদলাতে ও আর্কাইভ করতে পারবে (RBAC)।

#	বাটন / অংশ	API	ব্যাকএন্ডে কী ঘটবে (টেবিল)
1	**দাম বদলান**	PATCH `/products/:id/price`	নতুন দাম শুধু পরের বিক্রিতে; পুরনো ইনভয়েস অপরিবর্তিত। audit_logs: পুরনো→নতুন দাম, কে।
2	**স্টকের ইতিহাস**	UI → M19	
3	**তথ্য সম্পাদনা**	PATCH `/products/:id`	নাম, ক্যাটাগরি, সর্বনিম্ন স্টক ইত্যাদি। স্টক সংখ্যা এখানে বদলায় না।
4	**সক্রিয়/নিষ্ক্রিয়**	PATCH `/products/:id/status`	is_active; নিষ্ক্রিয় পণ্য বিক্রির তালিকায় আসে না।
5	**আর্কাইভ**	DELETE `/products/:id`	সফট ডিলিট (archived_at)। লেনদেন থাকলে মোছা যাবে না — শুধু আর্কাইভ; মালিকের নিশ্চিতকরণ লাগবে।



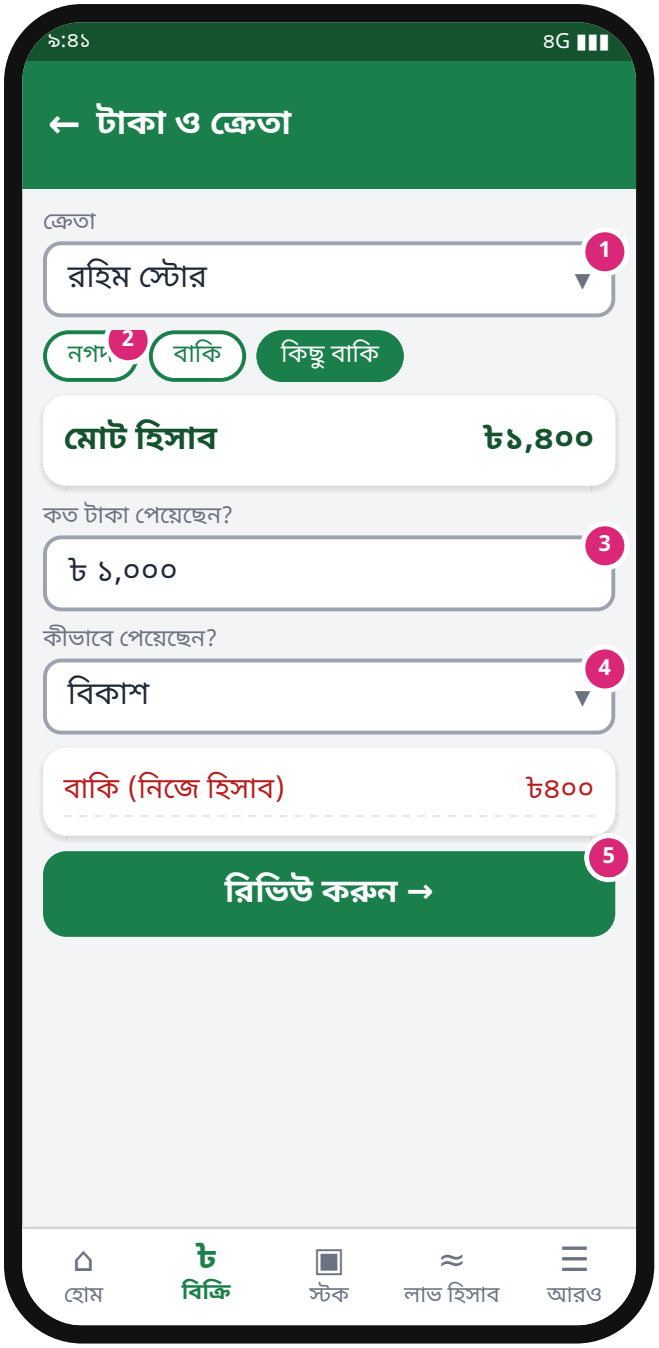
এই ধাপে এখনো সার্ভারে কিছু সেভ হয় না — কেবল ড্রাফট কার্ট।

#	বাটন / অংশ	API	ব্যাকএন্ডে কী ঘটবে (টেবিল)
1	**খোঁজার ঘর**	GET `/products?search=&active=true`	
2	**ক্যাটাগরি চিপ**	GET `/products?category_id=`	
3	**পণ্য কার্ড**	GET `/products/:id` (ক্যাশ)	ট্যাপে কার্টে ১ ইউনিট যোগ; দাম স্বয়ংক্রিয় products.selling_price থেকে। স্টক ০ হলে ধূসর + “স্টক নেই”।
4	**কার্ট দেখুন**	UI → M10	কার্ট ফোনে সংরক্ষিত থাকে (অসম্পূর্ণ ফর্ম হারাবে না)।



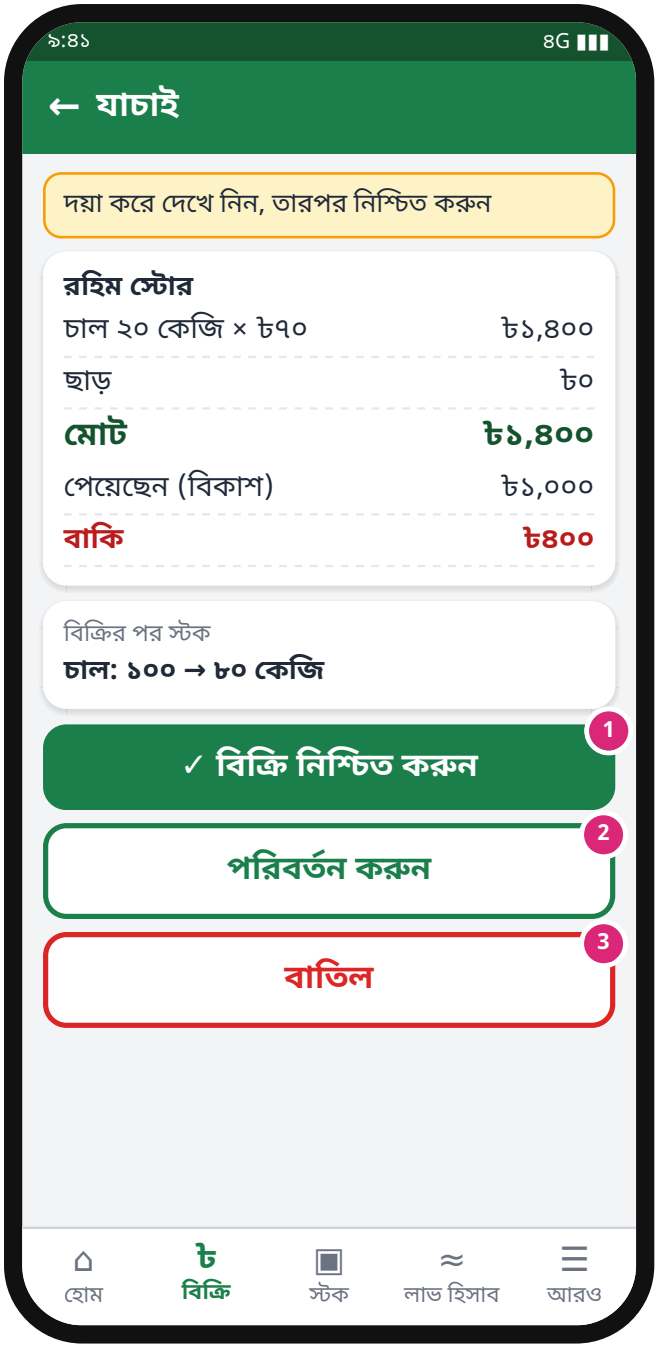
টাকার হিসাব সার্ভারে DECIMAL/মানি-সেফ লাইব্রেরিতে; সমান রাউন্ডিং নিয়ম সর্বত্র।

#	বাটন / অংশ	API	ব্যাকএন্ডে কী ঘটবে (টেবিল)
1	**– বাটন**	UI –	পরিমাণ কমে; ০ হলে পণ্য সরানোর প্রশ্ন।
2	**পরিমাণ ঘর (সংখ্যা কিপ্যাড)**	UI –	দশমিক চলবে (কেজি/লিটার)। স্টকের বেশি দিলে লাল সতর্কতা; negative stock বন্ধ থাকলে আটকাবে।
3	**+ বাটন**	UI –	
4	**ছাড়**	UI –	টাকা বা % বাছাই। ছাড় > উপমোট হলে ত্রুটি।
5	**আরও পণ্য**	UI → M09	একই বিক্রিতে একাধিক পণ্য।
6	**পরের ধাপ**	POST /sales/preview	সার্ভার সব মোট নতুন করে হিসাব করে ফেরত দেয় (ক্লায়েন্টের সংখ্যা বিশ্বাস করা হয় না)। কিছু সেভ হয় না।



বাকি টাকা ব্যবহারকারীকে হাতে হিসাব করতে হয় না।

#	বাটন / অংশ	API	ব্যাকএন্ডে কী ঘটবে (টেবিল)
1	**ক্রেতা বাছাই**	GET POST `/customers?search=`	তালিকা থেকে বাছা বা “+ নতুন” (নাম+ফোন)। “নগদ” বিক্রিতে ক্রেতা ঐচ্ছিক।
2	**নগদ/বাকি/কিছু বাকি**	UI –	নগদ = পুরো টাকা; বাকি = ০ পেয়েছে; কিছু বাকি = আংশিক। বাকি হলে ক্রেতা বাধ্যতামূলক।
3	**পাওয়া টাকা**	UI –	মোটের বেশি হলে আটকায়। বাকি = মোট − পাওয়া (সার্ভারে নিশ্চিত)।
4	**পেমেন্ট মাধ্যম**	GET `/payment-methods`	নগদ, বিকাশ, নগদ(MFS), ব্যাংক, কার্ড।
5	**রিভিউ**	POST `/sales/preview`	চূড়ান্ত হিসাব; এখনো সেভ নয়।



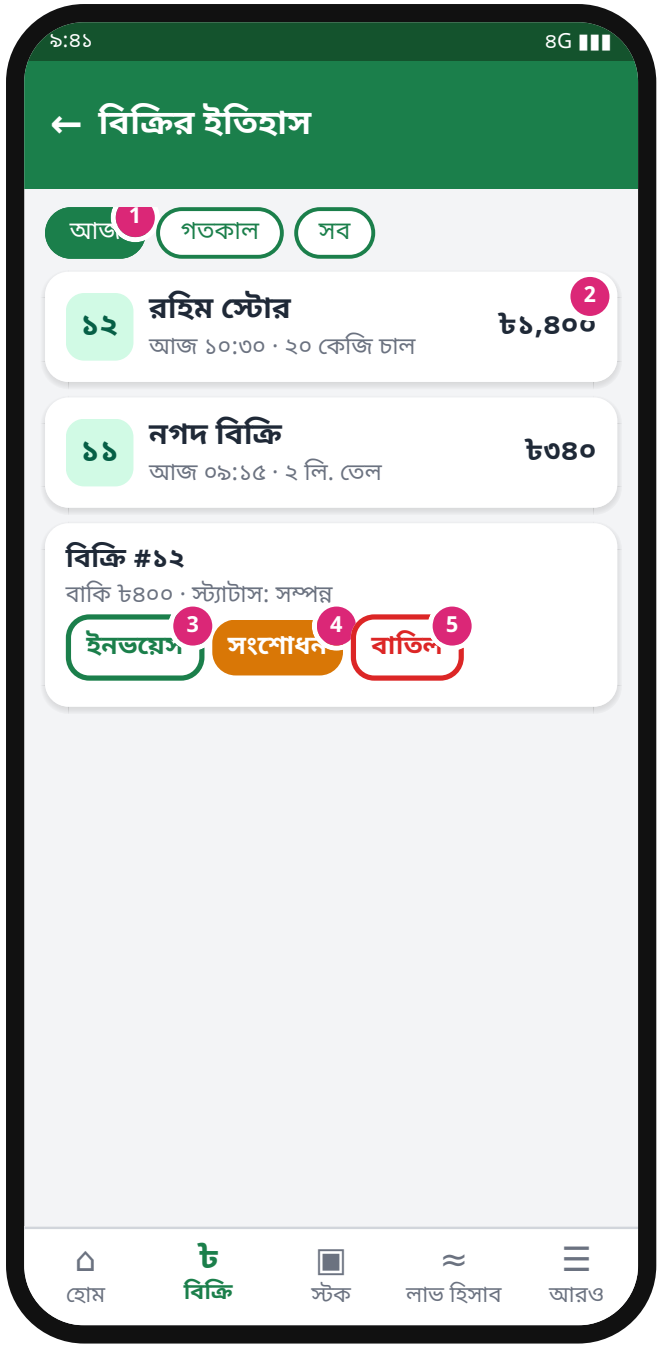
ডাবল-ট্যাপ বা খারাপ নেটে আবার পাঠালেও দুইবার বিক্রি হবে না — বাটন একবার চাপার পর অক্ষম হয়ে “সংরক্ষণ হচ্ছে…” দেখায়।

#	বাটন / অংশ	API	ব্যাকএন্ডে কী ঘটবে (টেবিল)
1	**বিক্রি নিশ্চিত**	`POST` `/sales (Header: Idempotency-Key)`	**এক ডাটাবেস ট্রানজ্যাকশনে:** sales + sale_items (তখনকার দাম ও কেনা দাম কপি) + inventory_movements(type=sale, −qty, source=sale_id) + products.current_stock কমে + payments (পাওয়া টাকা) + customers.balance বাড়ে + invoices (নম্বর অটো) + audit_logs। কোনো ধাপ ফেল করলে সব রোলব্যাক। একই Idempotency-Key দ্বিতীয়বার এলে নতুন লেনদেন হয় না, আগেরটাই ফেরত।
2	**পরিবর্তন করুন**	`UI` ← M10	
3	**বাতিল**	`UI` → M05	ড্রাফট কার্ট মুছে ফেলার আগে নিশ্চিতকরণ।



সফল বার্তা পরিষ্কার বাংলায়; ভুল হলে M44-এর ত্রুটি বার্তা।

#	বাটন / অংশ	API	ব্যাকএন্ডে কী ঘটবে (টেবিল)
1	**ইনভয়েস দেখুন/প্রিন্ট**	GET `/invoices/by-sale/:saleId`	সংরক্ষিত ইনভয়েস (লেনদেনের সময়ের দাম) → M35।
2	**আরেকটি বিক্রি**	UI → M09	নতুন Idempotency-Key তৈরি।
3	**হোমে যান**	GET `/reports/dashboard`	কার্ড রিফ্রেশ — নতুন বিক্রি, স্টক, পাওনা সব স্বয়ংক্রিয় বদলায়।



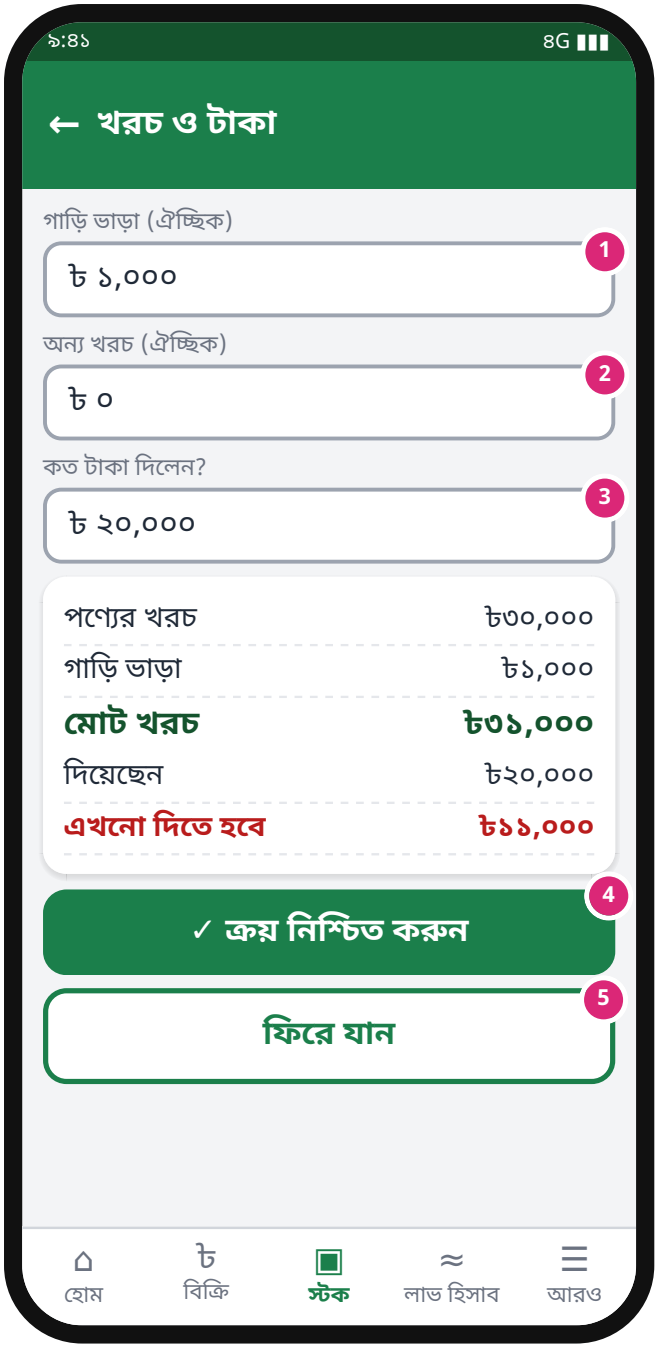
কোনো আর্থিক রেকর্ড ডিলিট হয় না — উল্টো এন্ট্রি ও স্ট্যাটাসে ট্রেস থাকে।

#	বাটন / অংশ	API	ব্যাকএন্ডে কী ঘটবে (টেবিল)
1	**তারিখ ফিল্টার**	GET `/sales?from&to&status`	
2	**বিক্রির সারি**	GET `/sales/:id`	আইটেমসহ বিস্তারিত।
3	**ইনভয়েস**	UI → M35	
4	**সংশোধন (শুধু Owner)**	PATCH `/sales/:id/correct`	সরাসরি এডিট নয়: কারণসহ সংশোধন → আগের প্রভাব উল্টে নতুন এন্ট্রি; inventory_movements-এ উল্টো ও নতুন মুভমেন্ট, ব্যালেন্স আবার হিসাব, audit_logs(আগের/পরের মান)।
5	**বাতিল (শুধু Owner)**	POST `/sales/:id/cancel`	কারণ বাধ্যতামূলক। status=cancelled; স্টক ফিরে আসে (reversal movement), ক্রেতার ব্যালেন্স ও পেমেন্ট উল্টায়; ইনভয়েসে “বাতিল” ছাপ। ডিলিট নয়।



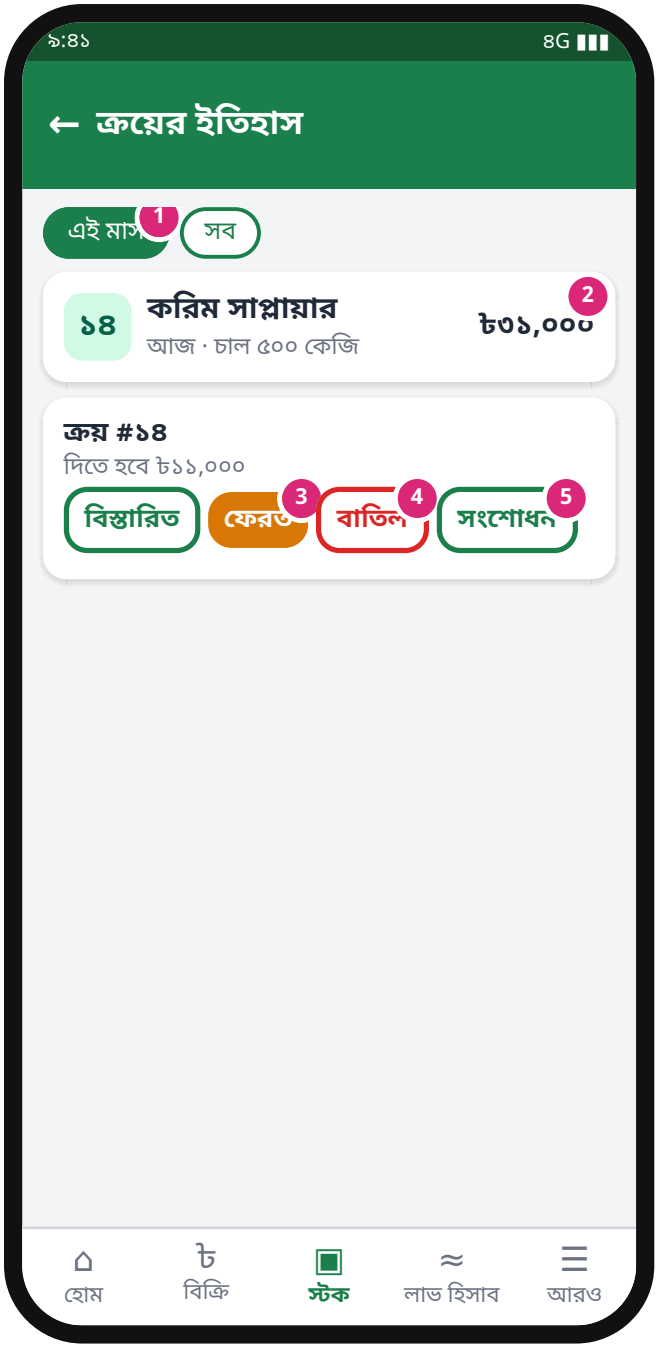
বিক্রির ধাঁচেই ধাপে ধাপে; ব্যবহারকারী মোট খরচ হাতে হিসাব করে না।

#	বাটন / অংশ	API	ব্যাকএন্ডে কী ঘটবে (টেবিল)
1	**সরবরাহকারী**	GET POST `/suppliers?search=`	তালিকা বা “+ নতুন”। পণ্যের পছন্দের সরবরাহকারী আগে থেকে বসে।
2	**পণ্য**	GET `/products`	—
3	**পরিমাণ**	UI –	
4	**কেনা দাম**	UI –	আগের কেনা দাম নিজে বসে; বদলানো যায়। ক্রয় নিশ্চিত হলে products.purchase_price হালনাগাদ হবে কিনা সেটি settings-এ।
5	**আরও পণ্য**	UI –	একাধিক purchase_items।
6	**পরের ধাপ**	POST `/purchases/preview`	সার্ভারে মোট হিসাব; কিছু সেভ নয়।

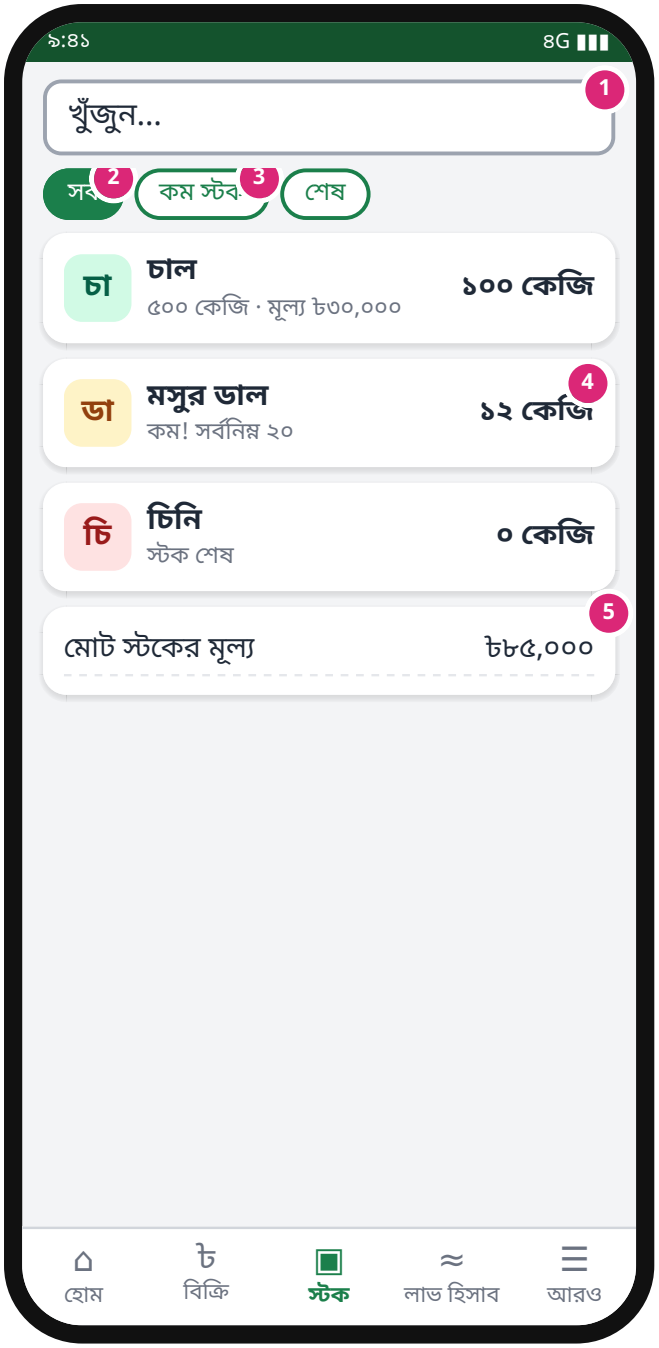


ক্রয়ের অতিরিক্ত খরচ কেনা দামে ভাগ হয়ে হিসাব হয় (পণ্যের প্রকৃত খরচ জানতে) — নিয়ম settings-এ।

#	বাটন / অংশ	API	ব্যাকএন্ডে কী ঘটবে (টেবিল)
1	**গাড়ি ভাড়া**	UI –	purchases.transport_cost
2	**অন্য খরচ**	UI –	purchases.other_cost
3	**দেওয়া টাকা**	UI –	মোটের বেশি হলে ত্রুটি।
4	**ক্রয় নিশ্চিত**	POST `/purchases (Idempotency-Key)`	**এক ট্রানজ্যাকশনে:** purchases + purchase_items + inventory_movements(type=purchase, +qty) + products.current_stock বাড়ে + payments (দেওয়া টাকা) + suppliers.balance বাড়ে (বাকি) + audit_logs। ফেল করলে সব রোলব্যাক।
5	**ফিরে যান**	UI ← M15	

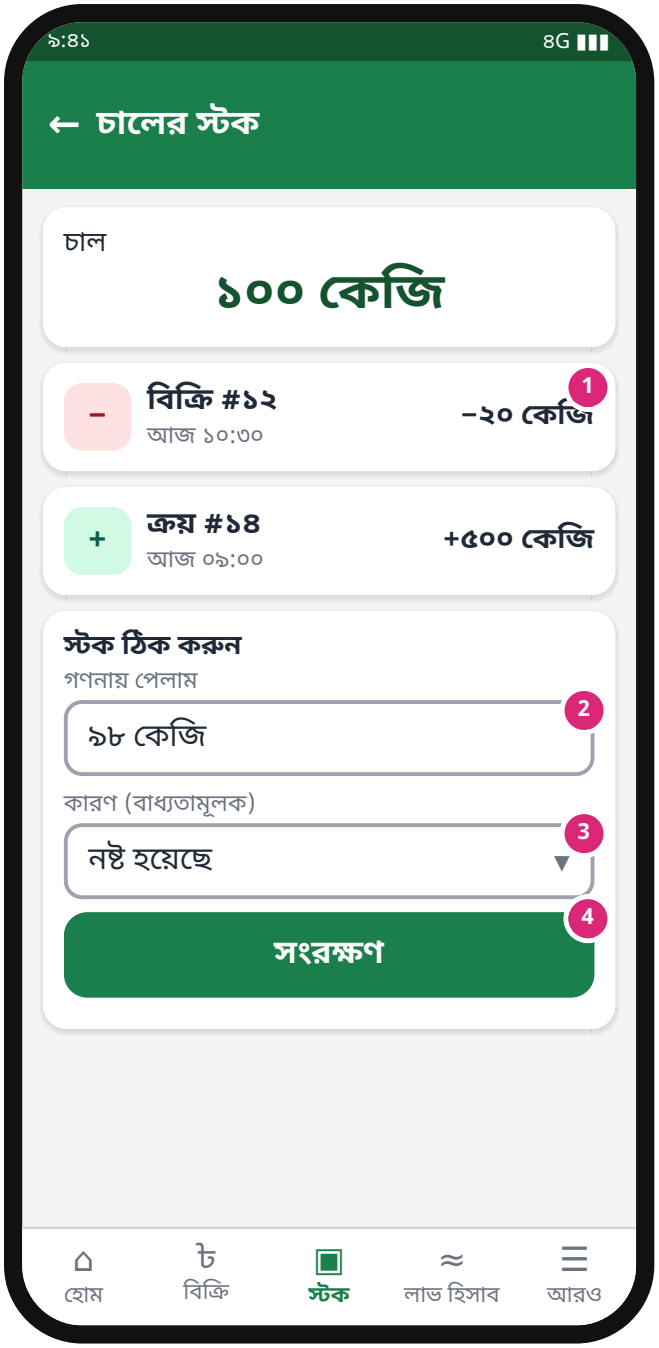


#	বাটন / অংশ	API	ব্যাকএন্ডে কী ঘটবে (টেবিল)
1	ফিল্টার	GET /purchases?from&to&supplier_id	
2	ক্রয়ের সারি	GET /purchases/:id	
3	ফেরত	POST /purchases/:id/return	ফেরত পণ্য ও পরিমাণ বাছুন → purchase_returns, স্টক কমে (return movement), সরবরাহকারীর ব্যালেন্স কমে/অগ্রিম হয়।
4	বাতিল	POST /purchases/:id/cancel	কারণ লাগবে; স্টক ও ব্যালেন্স উল্টায়; শুধু Owner। স্টক ইতিমধ্যে বিক্রি হয়ে নেগেটিভ হলে আটকাবে।
5	সংশোধন	PATCH /purchases/:id/correct	উল্টো + নতুন এন্ট্রি পদ্ধতি; audit_logs।



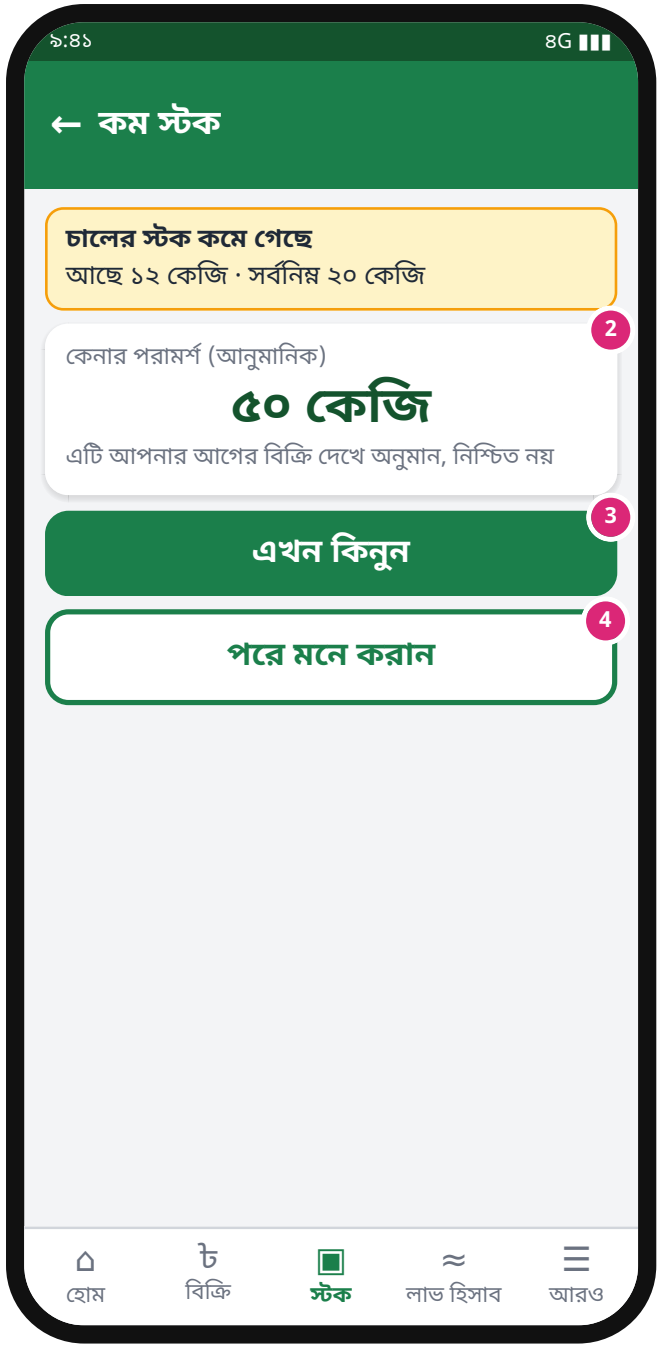
স্টক হাতে বদলানোর সুযোগ নেই; শুধু M19-এর “স্টক ঠিক করুন” কারণসহ।

#	বাটন / অংশ	API	ব্যাকএন্ডে কী ঘটবে (টেবিল)
1	**খোঁজা**	`GET /inventory?search=`	products.current_stock + মূল্য (stock × purchase_price)।
2	**সব**	`GET /inventory`	
3	**কম স্টক ফিল্টার**	`GET /inventory?low=true`	current_stock ≤ min_stock
4	**পণ্য ট্যাপ**	`UI → M19`	
5	**স্টকের মোট মূল্য**	`GET /reports/inventory/valuation`	Σ (স্টক × গড়/শেষ কেনা দাম) — পদ্ধতি settings-এ।



নেগেটিভ স্টক ডিফল্টে বন্ধ (settings.allow_negative_stock)।

#	বাটন / অংশ	API	ব্যাকএন্ডে কী ঘটবে (টেবিল)
1	**ইতিহাসের সারি**	GET `/inventory/:productId/movements`	প্রতিটি পরিবর্তনের উৎস: বিক্রি/ক্রয়/ফেরত/ঠিক/শুরুর স্টক + রেফারেন্স আইডি, ট্যাপে মূল লেনদেন।
2	**গণনায় পাওয়া স্টক**	UI –	সিস্টেম নিজে পার্থক্য (−২) হিসাব করে।
3	**কারণ**	GET `/inventory/adjustment-reasons`	নষ্ট, চুরি, গণনার ভুল, নমুনা, অন্য। কারণ ছাড়া সংরক্ষণ হবে না।
4	**সংরক্ষণ**	POST `/inventory/adjustments`	inventory_movements(type=adjustment, reason, user) + products.current_stock + audit_logs। শুধু অনুমতি থাকলে।



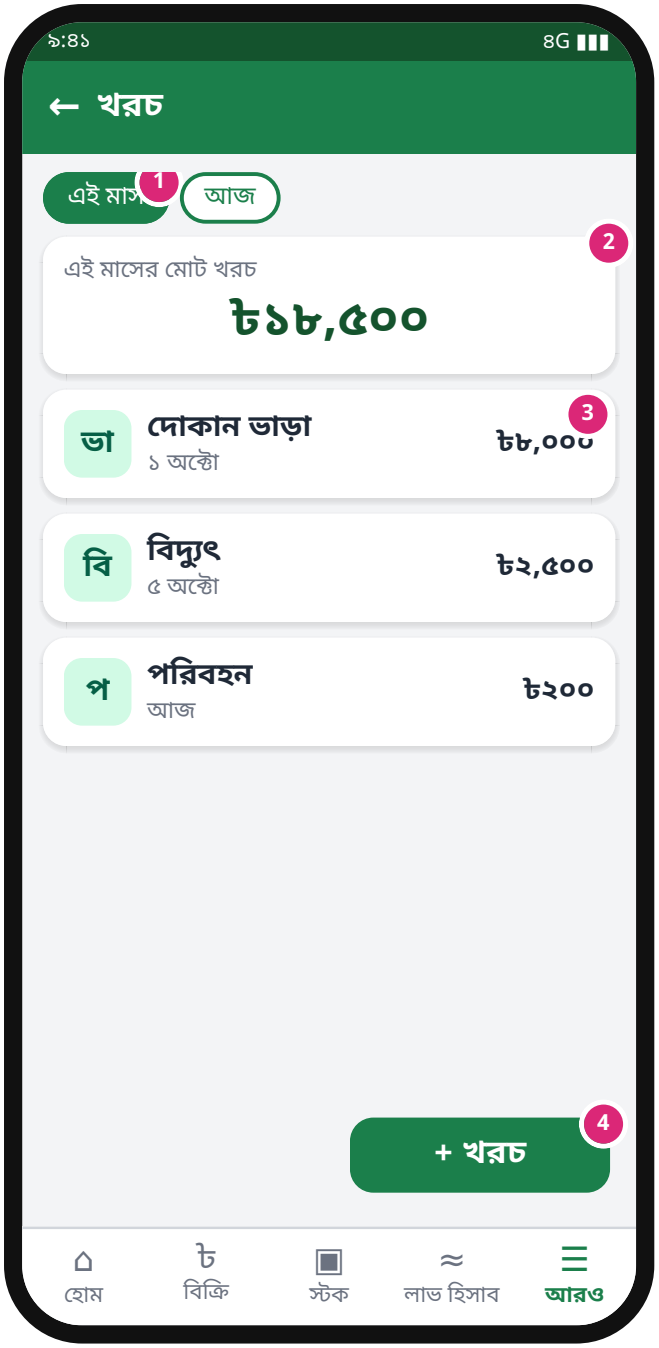
এটি পরামর্শ মাত্র — নিজে থেকে কিছু কেনা হয় না।

#	বাটন / অংশ	API	ব্যাকএন্ডে কী ঘটবে (টেবিল)
1	**সতর্কতা**	`GET /inventory/low-stock`	products.current_stock ≤ min_stock।
2	**কেনার পরামর্শ**	`GET /inventory/:id/suggestion`	গত ৩০ দিনের গড় বিক্রি × ১৫ দিন − বর্তমান স্টক; সবসময় “আনুমানিক” লেবেলসহ (ডেটা কম হলে দেখায় না)।
3	**এখন কিনুন**	`UI → M15`	পণ্য ও পরিমাণ আগে থেকে বসানো; মানুষ বদলাতে পারে।
4	**পরে মনে করান**	`POST /inventory/:id/snooze`	products.alert_snoozed_until (Phase 2-এ নোটিফিকেশন)।

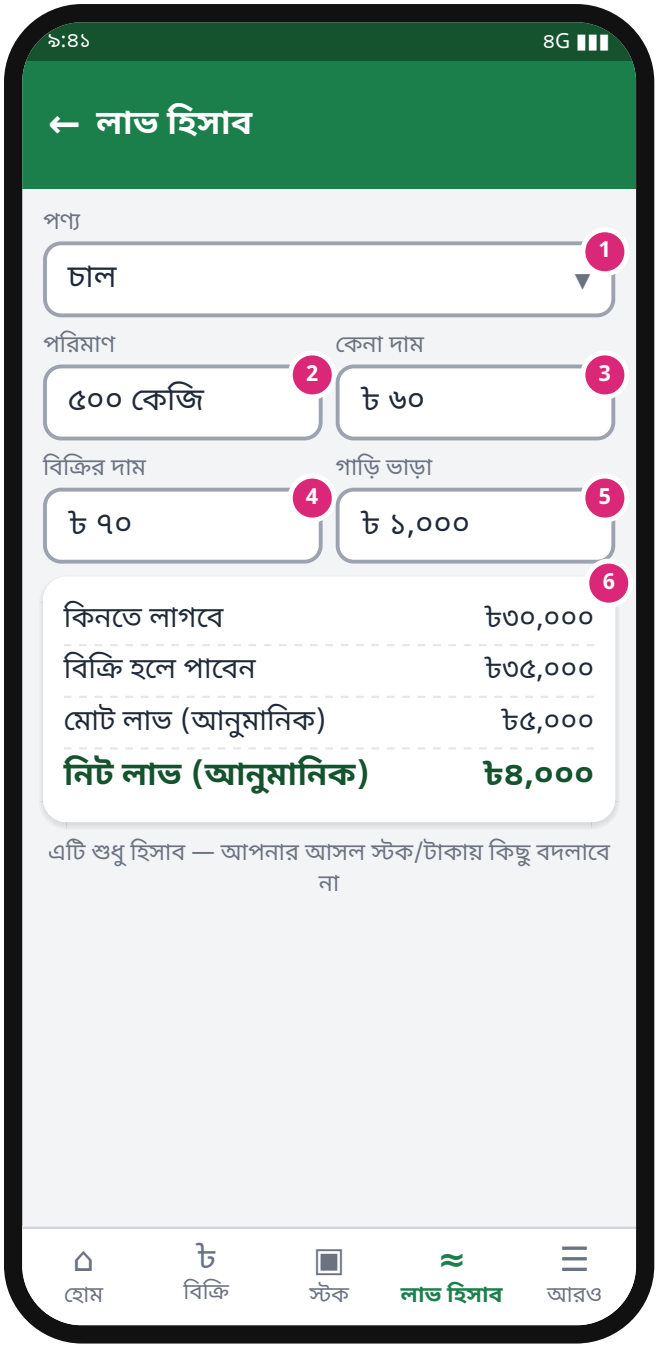


নোট ও রসিদ “আরও তথ্য”-এর ভেতরে, ডিফল্টে লুকানো।

#	বাটন / অংশ	API	ব্যাকএন্ডে কী ঘটবে (টেবিল)
1	**খরচের ধরন**	GET `/expenses/categories`	ভাড়া, বিদ্যুৎ, বেতন, পরিবহন, প্যাকেজিং, মেরামত, ইন্টারনেট, ফোন, অন্যান্য। নিজের ধরনও যোগ যায়।
2	**টাকা**	UI –	> ০ হতে হবে।
3	**তারিখ**	UI –	ডিফল্ট আজ; ভবিষ্যৎ তারিখে সতর্কতা।
4	**পেমেন্ট মাধ্যম**	GET `/payment-methods`	
5	**সংরক্ষণ**	POST `/expenses (Idempotency-Key)`	expenses + payments(out) + audit_logs। রসিদের ছবি ঐচ্ছিক (পরে)। এটি Net Profit-এ বাদ যায়, Gross Profit-এ নয়।



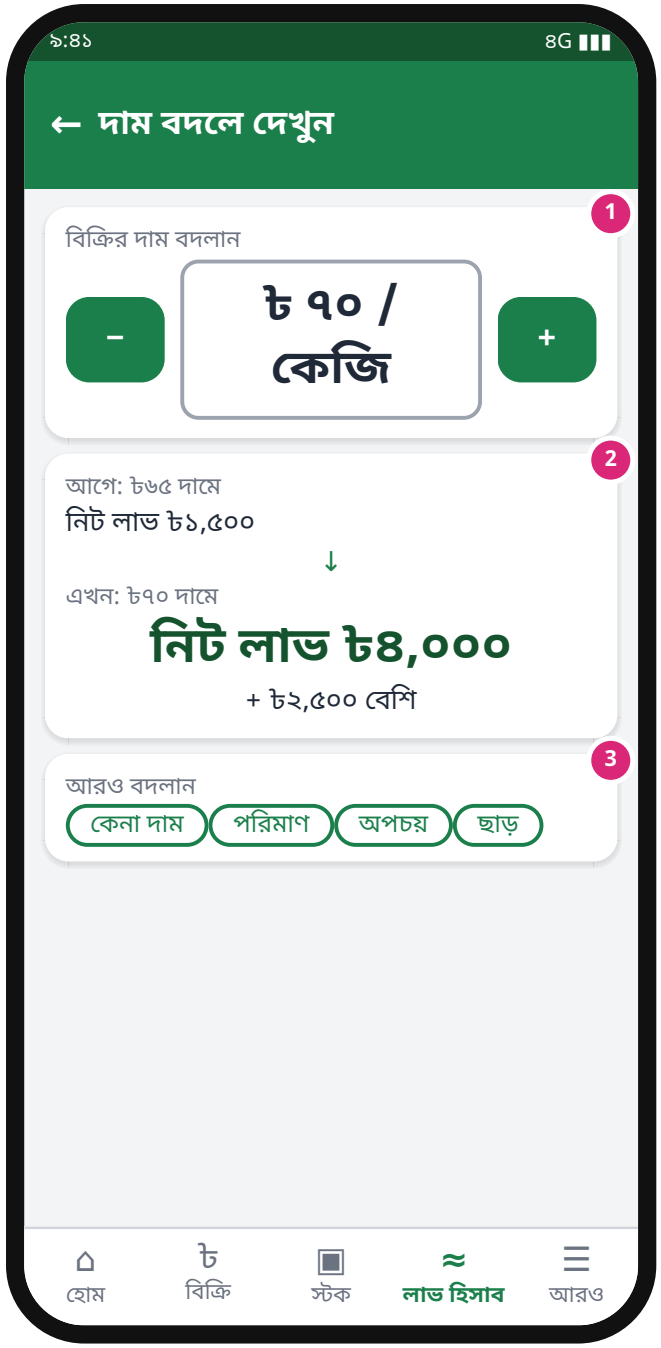
#	বাটন / অংশ	API	ব্যাকএন্ডে কী ঘটবে (টেবিল)
1	সময় ফিল্টার	GET /expenses?from&to	
2	মোট খরচ	GET /reports/expenses/summary?group=category	ধরন অনুযায়ী ভাগ → M37।
3	খরচের সারি	PATCH DELETE /expenses/:id	ভুল হলে সংশোধন/বাতিল (Owner), কারণসহ audit_logs।
4	+ খরচ	UI → M21	



সূত্র: কেনা খরচ = পরিমাণ×কেনা দাম; মোট লাভ = বিক্রি − কেনা খরচ; নিট লাভ = মোট লাভ − গাড়ি ও অন্য খরচ।

#	বাটন / অংশ	API	ব্যাকএন্ডে কী ঘটবে (টেবিল)
1	**পণ্য (ঐচ্ছিক বাছা)**	GET `/products`	বাছলে কেনা/বিক্রির দাম নিজে আসে।
2	**পরিমাণ**	UI –	**প্রতি অক্ষরে তাৎক্ষণিক হিসাব**: ক্লায়েন্টে লোকাল ফাংশন (shared/estimator.ts), তাই ইন্টারনেট ছাড়াই চলে।
3	**কেনা দাম**	UI –	
4	**বিক্রির দাম**	UI –	
5	**গাড়ি/অন্য খরচ, অপচয়, ছাড়**	UI –	ঐচ্ছিক ঘর।
6	**ফলাফল কার্ড**	POST `/estimator/calculate` (ঐচ্ছিক)	একই ফর্মুলা সার্ভারেও (যাচাই/সেভের জন্য)। **কখনো products, sales, purchases, balances ছোঁয় না**; "সংরক্ষণ" করলে শুধু estimates টেবিলে যায়।

এই পাতার পেছনের পরীক্ষা: ৫০০ কেজি × (৭০−৬০) = ৫,০০০; গাড়ি ১,০০০ বাদে নিট ৪,০০০ ✓



এখানে কোনো সার্ভার কল দরকার নেই — তাই ধীর নেটেও দ্রুত।

#	বাটন / অংশ	API	ব্যাকএন্ডে কী ঘটবে (টেবিল)
1	**+/− বা সংখ্যা**	UI –	প্রতিটি ধাপে তখনই ফলাফল বদলায় (লোকাল গণনা)।
2	**আগে/এখন তুলনা**	UI –	আগের ফলাফল ধরে রেখে পার্থক্য দেখায়।
3	**অন্য বিষয় বদল**	UI –	কেনা দাম, পরিমাণ, গাড়ি ভাড়া, অপচয়%, ছাড়, বিক্রির পরিমাণ।



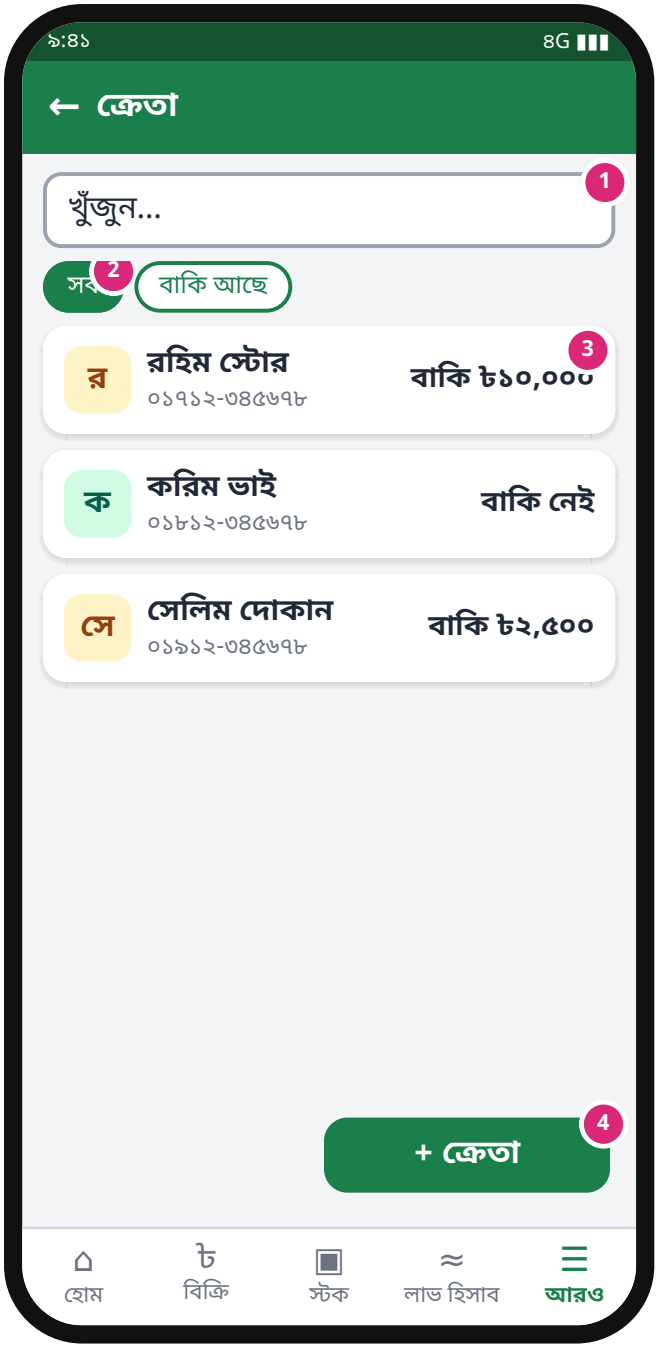
বিনিয়োগকারী ডেমোর ধাপ ৫-৬ এখান থেকে দেখানো হবে।

#	বাটন / অংশ	API	ব্যাকএন্ডে কী ঘটবে (টেবিল)
1	**তিন কলাম**	UI –	প্রতি কলামে নিজস্ব ইনপুট; সবই লোকাল গণনা।
2	**লোকসান সতর্কতা**	UI –	নিট লাভ < ০ হলে লাল।
3	**সংরক্ষণ**	POST `/estimator/scenarios`	estimates + estimate_scenarios (আলাদা টেবিল, আসল ডেটার সাথে যুক্ত নয়)।
4	**PDF/শেয়ার**	GET `/estimator/scenarios/:id/pdf`	বিনিয়োগকারীকে দেখানোর জন্য।

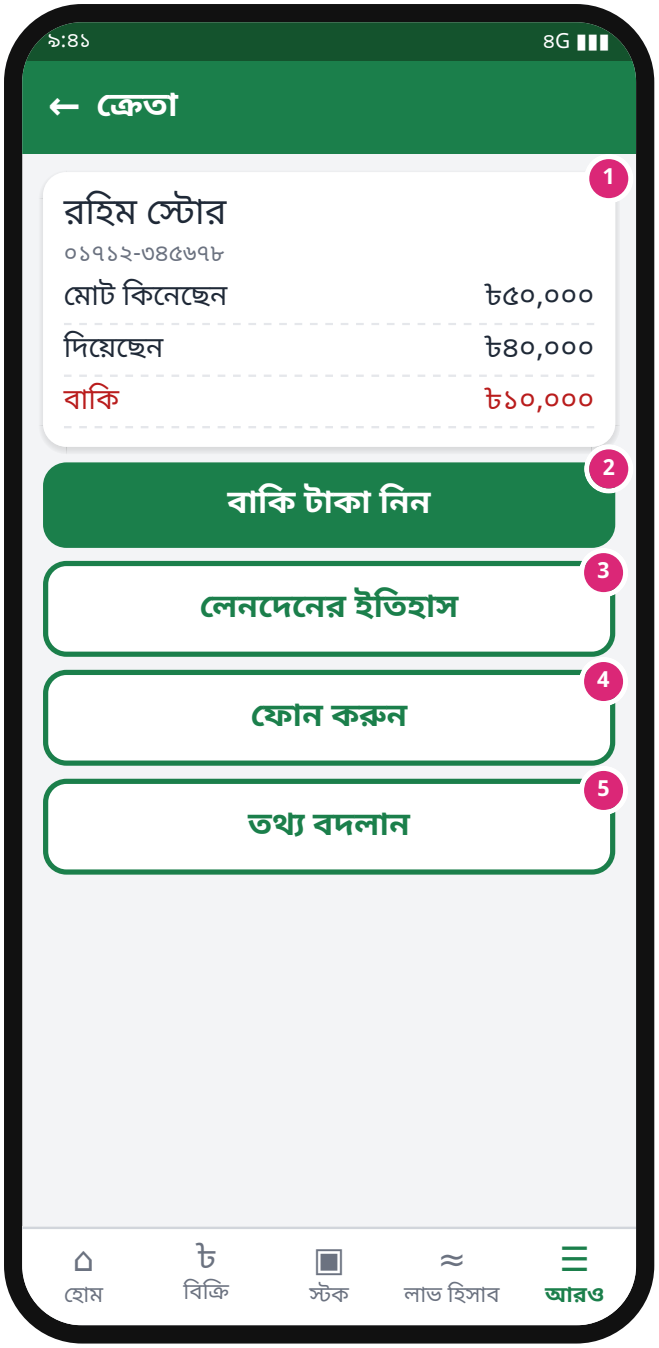


মূল মেনু ৫টি: হোম, বিক্রি, স্টক, লাভ হিসাব, আরও।

#	বাটন / অংশ	API	ব্যাকএন্ডে কী ঘটবে (টেবিল)
1	ক্রেতা	UI → M27	
2	সরবরাহকারী	UI → M31	
3	বাকি-পাওনা	UI → M33	
4	ইনভয়েস	UI → M34	
5	খরচ	UI → M22	
6	রিপোর্ট	UI → M36	
7	সেটিংস	UI → M40	শুধু Owner
8	ব্যবহারকারী	UI → M41	শুধু Owner (RBAC: Staff-এর কাছে বাটন লুকানো, সার্ভারেও আটকানো)।

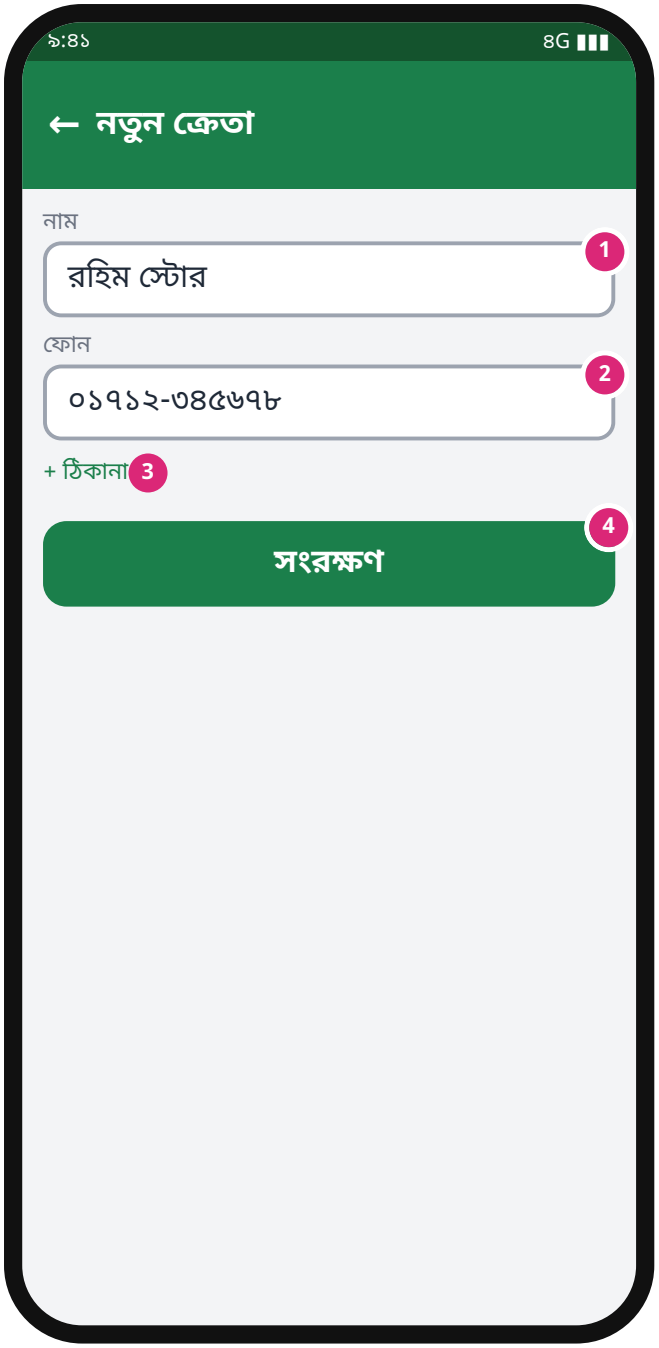


#	বাটন / অংশ	API	ব্যাকএন্ডে কী ঘটবে (টেবিল)
1	খোঁজা	GET /customers?search=	নাম/ফোন
2	শুধু বাকি আছে	GET /customers?has_due=true	balance > 0
3	ক্রেতা সারি	GET /customers/:id	→ M28
4	+ ক্রেতা	UI → M29	

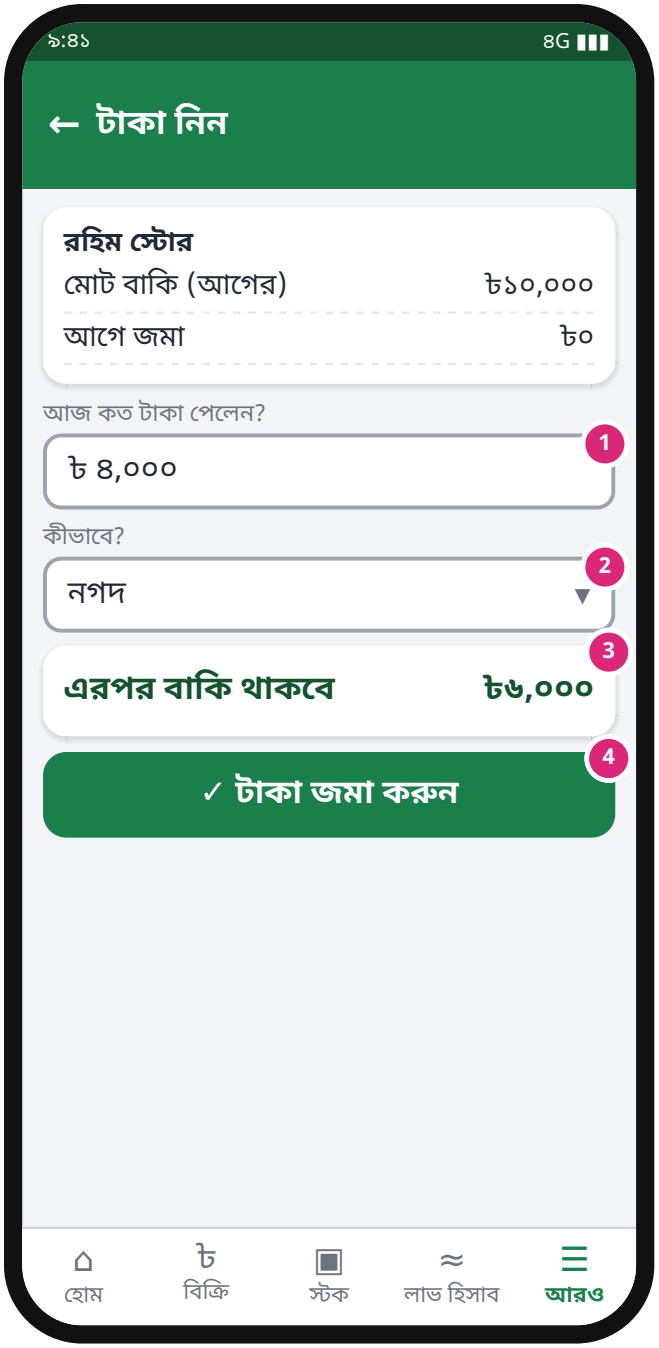


এটাই আপনার দেওয়া নমুনা: কেনা ৫০,০০০, দিয়েছেন ৪০,০০০, বাকি ১০,০০০।

#	বাটন / অংশ	API	ব্যাকএন্ডে কী ঘটবে (টেবিল)
1	**তিনটি সংখ্যা**	GET `/customers/:id/summary`	মোট কেনা = Σ sales.total (বাতিল বাদে); দিয়েছেন = Σ payments; বাকি = কেনা − দেওয়া (সংরক্ষিত balance-এর সাথে যাচাই)।
2	**বাকি টাকা নিন**	UI → M30	
3	**লেনদেনের ইতিহাস**	GET `/customers/:id/transactions`	বিক্রি ও পেমেন্ট মিশিয়ে সময়ক্রমে।
4	**ফোন করুন**	UI `tel:`	ফোনের ডায়ালার খোলে।
5	**তথ্য বদলান**	PATCH `/customers/:id`	নাম, ফোন, ঠিকানা (ঐচ্ছিক)।

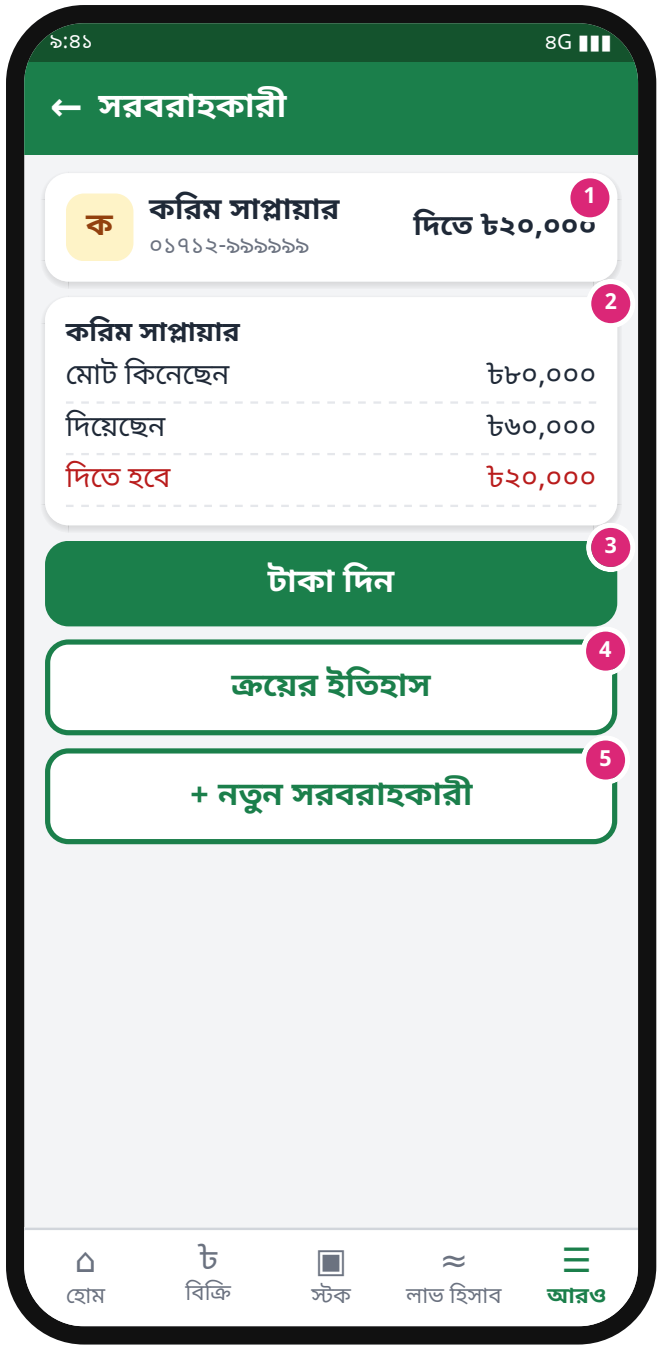


#	বাটন / অংশ	API	ব্যাকএন্ডে কী ঘটবে (টেবিল)
1	**নাম**	UI –	বাধ্যতামূলক
2	**ফোন**	UI –	ইউনিক (ব্যবসা ভিত্তিক); দ্বিতীয়বার দিলে “এই ক্রেতা আছে” সতর্কতা।
3	**ঠিকানা**	UI –	ঐচ্ছিক
4	**সংরক্ষণ**	POST `/customers`	customers + audit_logs; শুরুর বাকি ঐচ্ছিক (opening balance)।



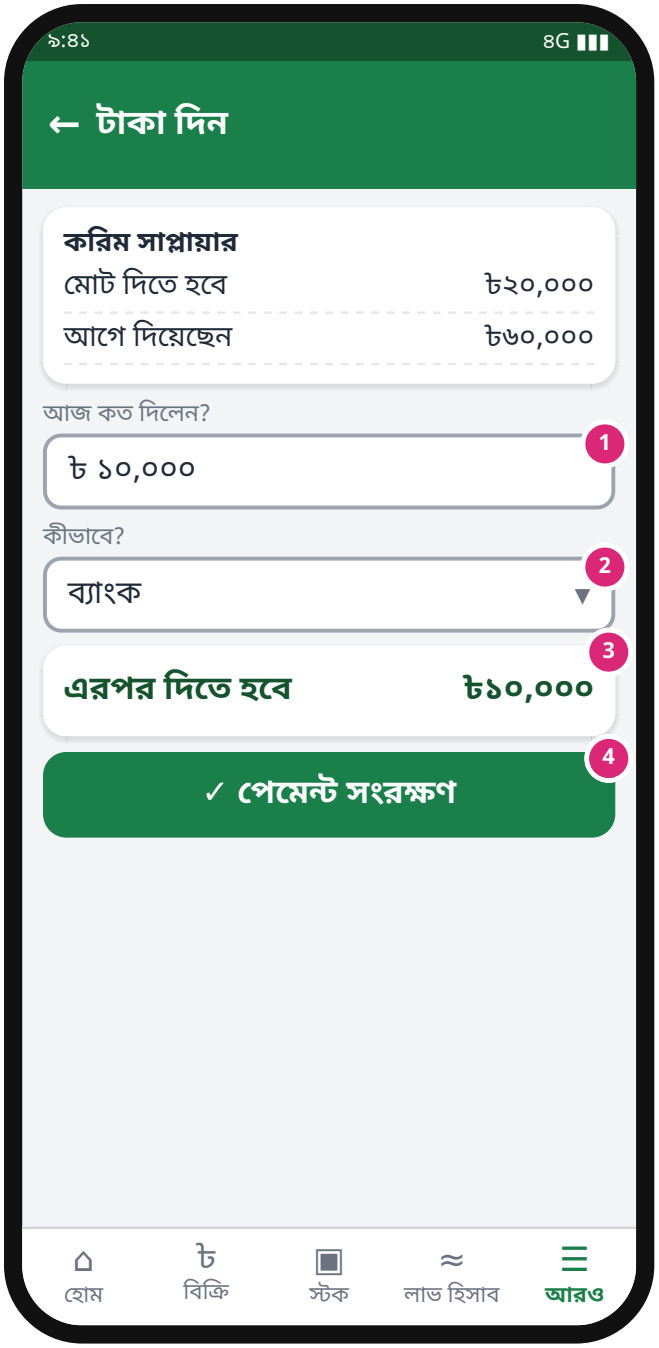
প্রতিটি পেমেন্টে চারটি সংখ্যা দেখানো বাধ্যতামূলক: মূল / আগে দেওয়া / নতুন / বাকি।

#	বাটন / অংশ	API	ব্যাকএন্ডে কী ঘটবে (টেবিল)
1	**পাওয়া টাকা**	UI –	বাকির বেশি হলে আটকায়/অগ্রিম হিসেবে জিজ্ঞাসা।
2	**মাধ্যম**	GET `/payment-methods`	
3	**বাকি থাকবে (অটো)**	POST `/payments/receive/preview`	মূল পরিমাণ, আগে দেওয়া, নতুন, বাকি — চারটিই দেখায়।
4	**টাকা জমা**	POST `/payments/receive (Idempotency-Key)`	payments + customers.balance কমে + (নির্দিষ্ট বিক্রির বিপরীতে হলে sales.paid বাড়ে) + audit_logs। একই কী/একই পরিমাণ ৩০ সেকেন্ডে দ্বিতীয়বার এলে “ডুপ্লিকেট?” সতর্কতা।

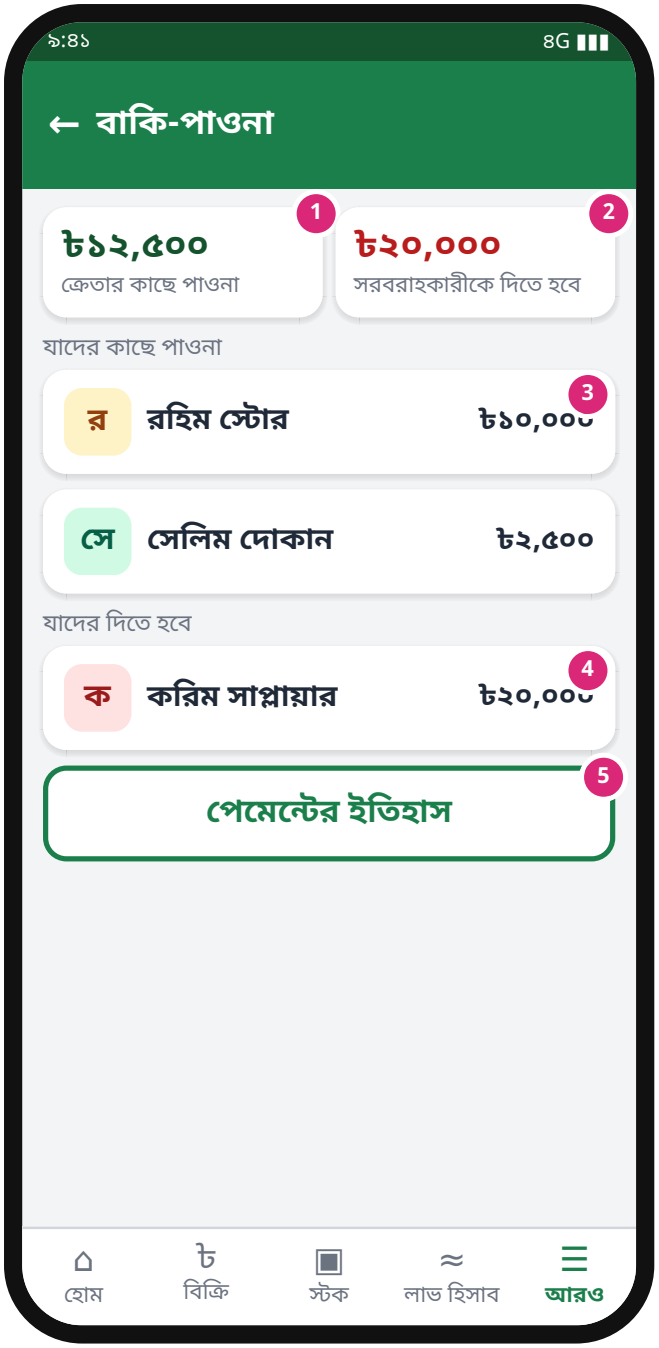


আপনার নমুনা: কেনা ৮০,০০০, দেওয়া ৬০,০০০, দিতে হবে ২০,০০০।

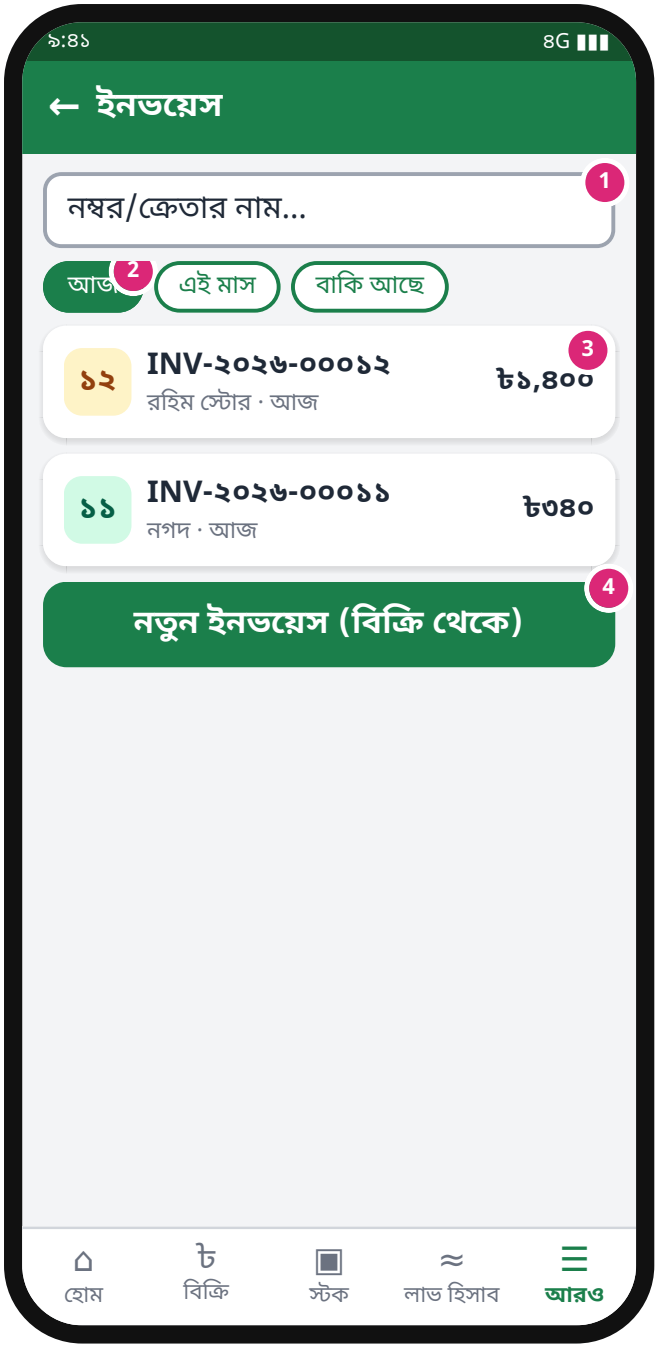
#	বাটন / অংশ	API	ব্যাকএন্ডে কী ঘটবে (টেবিল)
1	সারি	GET /suppliers?search=	
2	তিন সংখ্যা	GET /suppliers/:id/summary	মোট ক্রয় = Σ purchases.total; দেওয়া = Σ payments(out); দিতে হবে = পার্থক্য।
3	টাকা দিন	UI → M32	
4	ক্রয়ের ইতিহাস	GET /suppliers/:id/purchases	
5	নতুন সরবরাহকারী	POST /suppliers	নাম, ফোন, ঠিকানা।



#	বাটন / অংশ	API	ব্যাকএন্ডে কী ঘটবে (টেবিল)
1	টাকা	UI –	
2	মাধ্যম	GET /payment-methods	
3	অটো বাকি	POST /payments/pay/preview	
4	সংরক্ষণ	POST /payments/pay (Idempotency-Key)	payments(out) + suppliers.balance কমে + (নির্দিষ্ট ক্রয়ে হলে purchases.paid) + audit_logs।



#	বাটন / অংশ	API	ব্যাকএন্ডে কী ঘটবে (টেবিল)
1	মোট পাওনা	GET /payments/receivables/summary	
2	মোট দেনা	GET /payments/payables/summary	
3	ক্রেতা সারি	UI → M28 / M30	
4	সরবরাহকারী সারি	UI → M31 / M32	
5	পেমেন্টের ইতিহাস	GET /payments?type=in|out&from&to	সব পেমেন্ট: কে, কত, কবে, কোন লেনদেনের বিপরীতে।



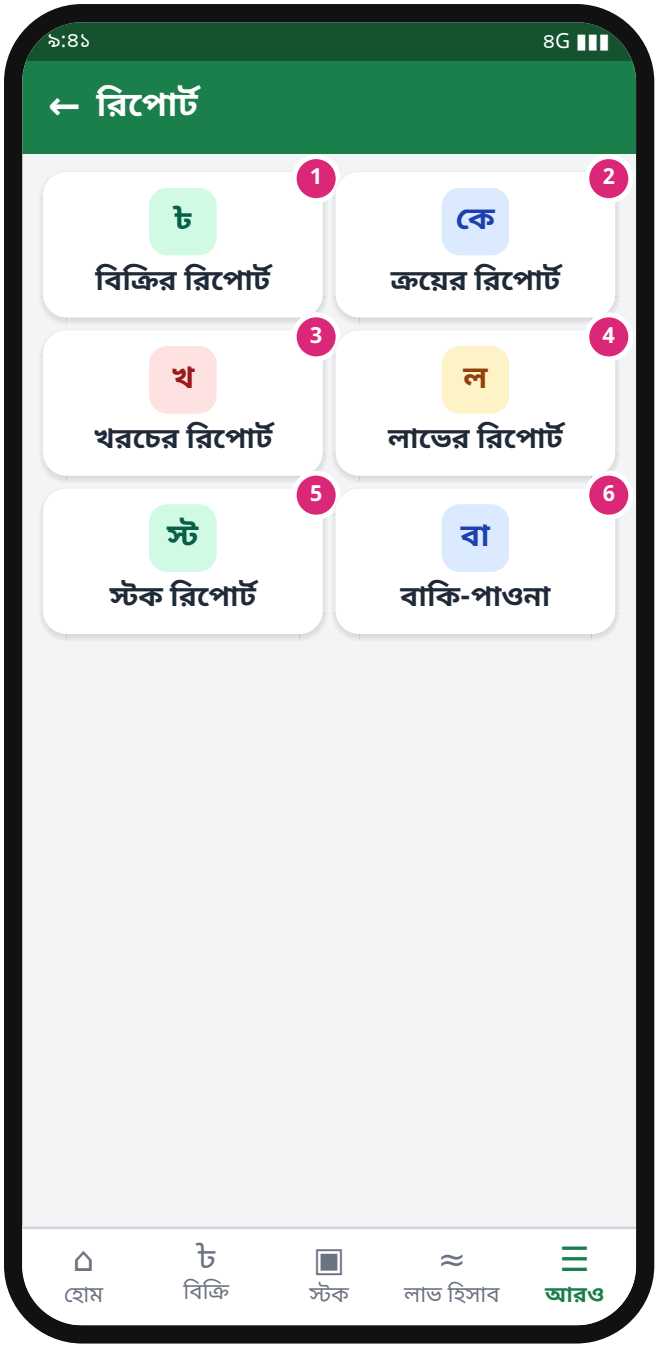
#	বাটন / অংশ	API	ব্যাকএন্ডে কী ঘটবে (টেবিল)
1	**খোঁজা**	GET `/invoices?search=`	নম্বর, ক্রেতা, তারিখ।
2	**ফিল্টার**	GET `/invoices?status=due|paid`	
3	**ইনভয়েস সারি**	GET `/invoices/:id`	→ M35
4	**নতুন ইনভয়েস**	UI → `M09`	ইনভয়েস শুধু সম্পন্ন বিক্রি থেকেই স্বয়ংক্রিয় তৈরি হয় (হাতে বানানো নয়)।



ইনভয়েসে লেনদেনের মুহূর্তের নাম ও দাম sale_items-এ কপি করা; আজ পণ্যের দাম বদলালে পুরনো ইনভয়েস বদলায় না।

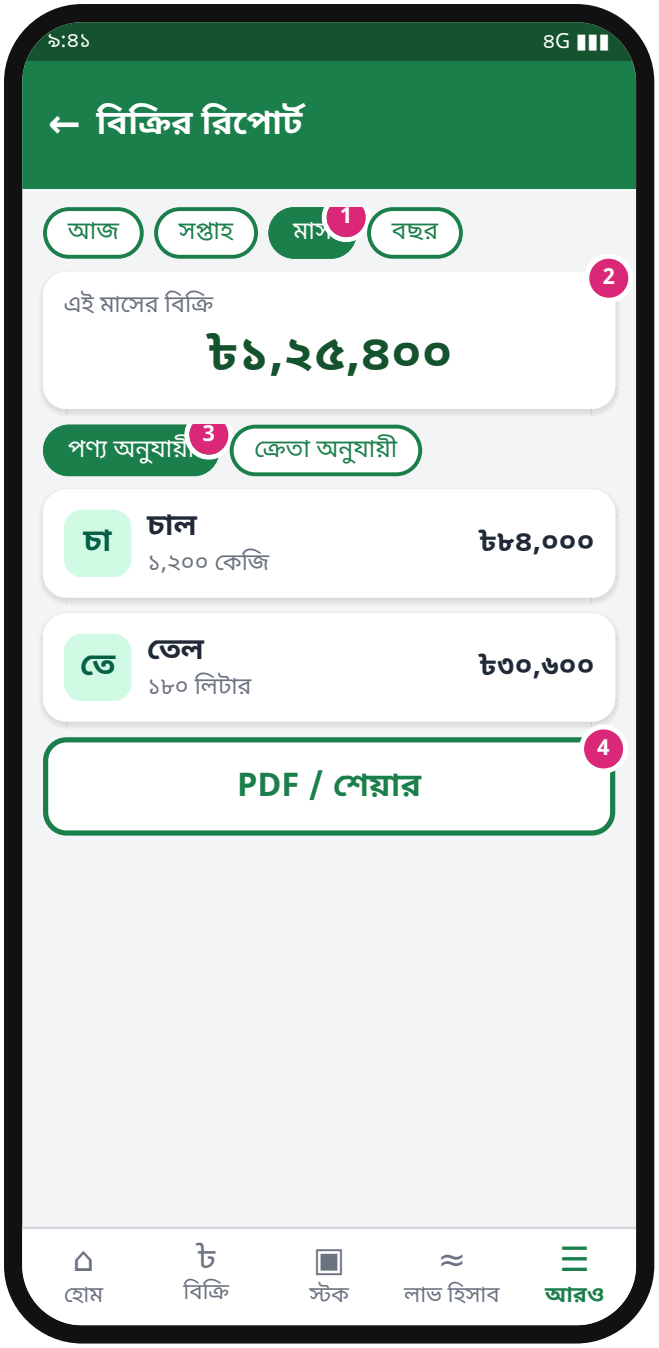
#	বাটন / অংশ	API	ব্যাকএন্ডে কী ঘটবে (টেবিল)
1	**ভাষা বাংলা/English**	GET `/invoices/:id?lang=bn|en`	একই ডেটা, ভিন্ন লেবেল।
2	**প্রিন্ট / পুনরায় প্রিন্ট**	POST `/invoices/:id/print`	প্রিন্ট কাউন্ট ও তারিখ invoices.reprint_count-এ; ফোনের প্রিন্ট ডায়ালগ/ব্লুটুথ প্রিন্টার।
3	**PDF**	GET `/invoices/:id/pdf`	সার্ভারে PDF তৈরি; WhatsApp/ইমেইলে শেয়ার।

ইনভয়েসের তথ্য: ব্যবসার নাম, যোগাযোগ, ইনভয়েস নম্বর, তারিখ, ক্রেতা, পণ্য, পরিমাণ, একক দাম, লাইন মোট, ছাড়, অতিরিক্ত চার্জ, মোট, পেয়েছি, বাকি, পরিশোধের অবস্থা।

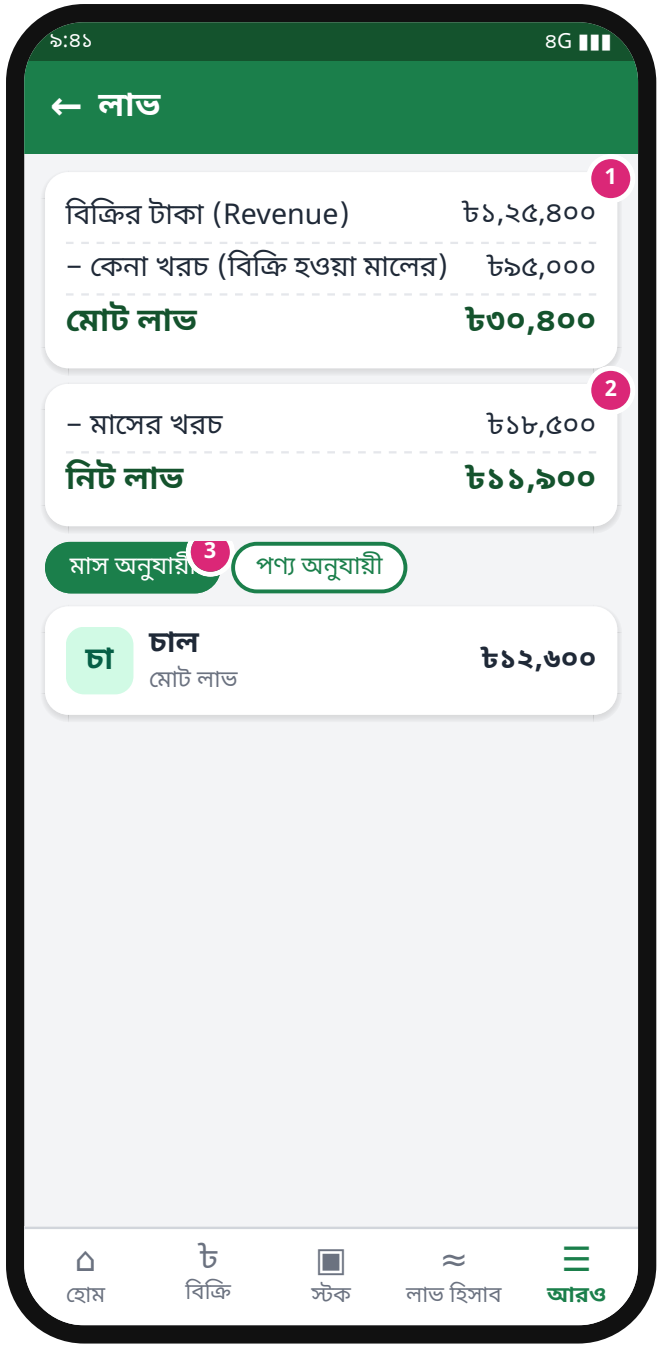


সব রিপোর্ট সংরক্ষিত লেনদেন থেকে এক্ষুনি হিসাব হয় — আলাদা "রিপোর্ট টেবিল" হাতে বদলানো হয় না।

#	বাটন / অংশ	API	ব্যাকএন্ডে কী ঘটবে (টেবিল)
1	বিক্রি	UI → M37	
2	ক্রয়	GET /reports/purchases?group=day|week|month|supplier|product	
3	খরচ	GET /reports/expenses?group=category|day|month	
4	লাভ	UI → M38	
5	স্টক	GET /reports/inventory?type=current|low|value|movement	
6	বাকি-পাওনা	GET /reports/dues?type=customer|supplier|history	

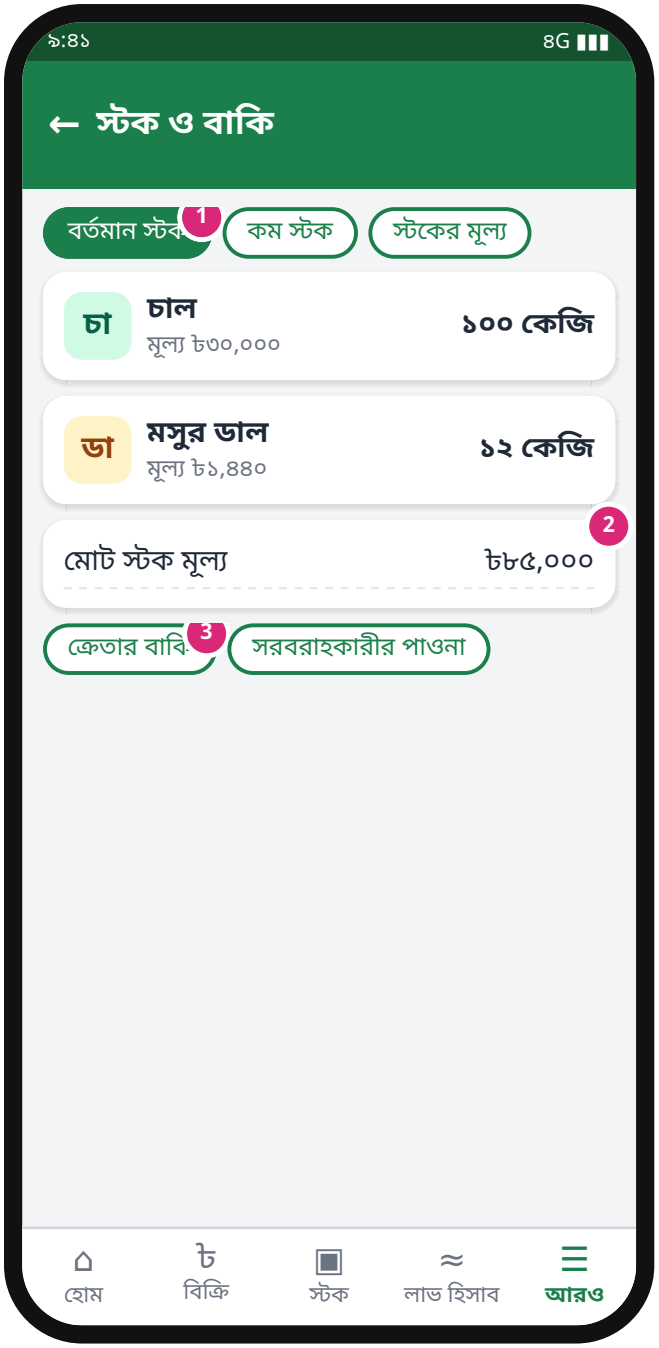


#	বাটন / অংশ	API	ব্যাকএন্ডে কী ঘটবে (টেবিল)
1	**সময়**	GET `/reports/sales?period=day|week|month|year`	
2	**মোট বিক্রি**	GET (ওপরের API)	sales.total যোগফল — এটি “বিক্রির টাকা”, লাভ নয়।
3	**পণ্য/ক্রেতা অনুযায়ী**	GET `/reports/sales?group=product|customer`	sale_items ও sales join।
4	**PDF/শেয়ার**	GET `/reports/sales/pdf`	

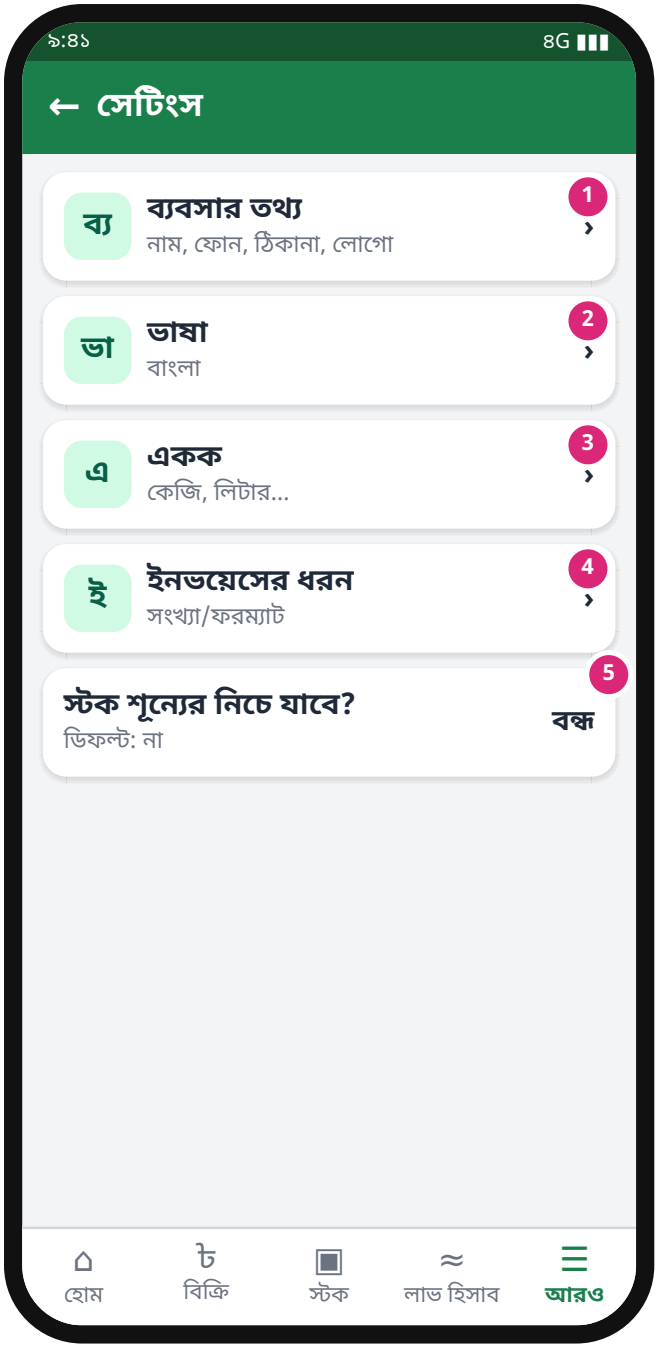


কখনো “বিক্রির টাকা”কে লাভ বলা হবে না — আলাদা লেবেল।

#	বাটন / অংশ	API	ব্যাকএন্ডে কী ঘটবে (টেবিল)
1	**মোট লাভ (Gross)**	GET `/reports/profit?type=gross`	= বিক্রির টাকা − বিক্রি হওয়া পণ্যের কেনা খরচ (COGS)। COGS আসে sale_items.cost_price_snapshot থেকে।
2	**নিট লাভ (Net)**	GET `/reports/profit?type=net`	= মোট লাভ − খরচ (expenses)।
3	**মাস/পণ্য অনুযায়ী**	GET `/reports/profit?group=month|product`	

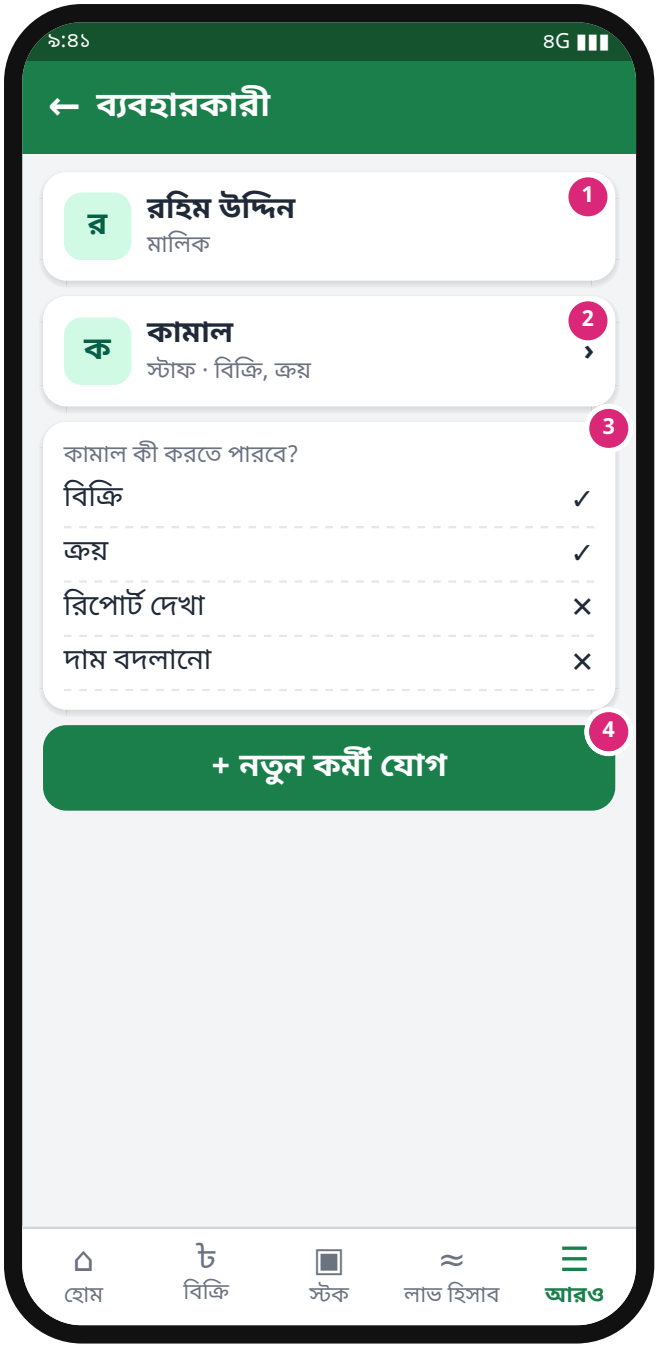


#	বাটন / অংশ	API	ব্যাকএন্ডে কী ঘটবে (টেবিল)
1	**স্টক ফিল্টার**	GET `/reports/inventory?type=current|low|value|movement`	
2	**মোট মূল্য**	GET `/reports/inventory/valuation`	
3	**বাকির তালিকা**	GET `/reports/dues?type=customer|supplier`	payments_history ও আছে।



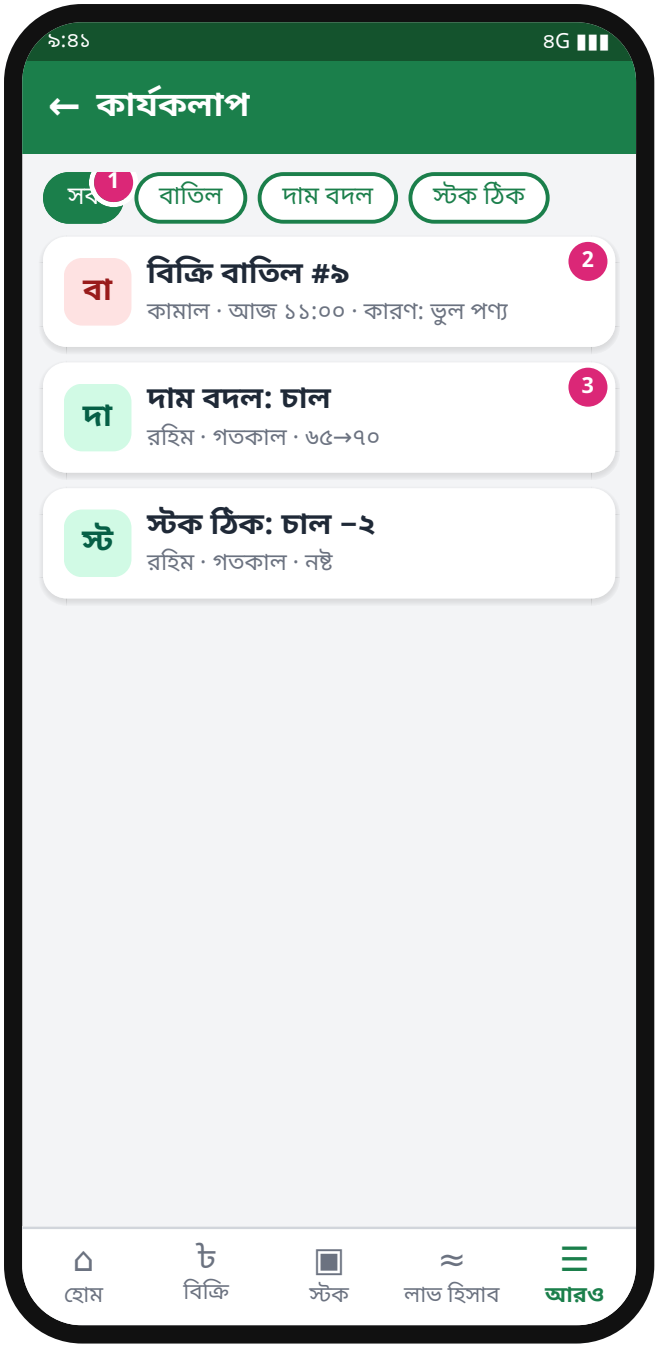
শুধু Owner ঢুকতে পারবে।

#	বাটন / অংশ	API	ব্যাকএন্ডে কী ঘটবে (টেবিল)
1	**ব্যবসার তথ্য**	PATCH `/settings/business`	businesses নাম, ফোন, ঠিকানা, লোগো; ইনভয়েসে ছাপা হয়।
2	**ভাষা**	PATCH `/settings/language`	settings.language = bn|en
3	**একক**	GET POST `/units`	কাস্টম একক যোগ।
4	**ইনভয়েস ফরম্যাট**	PATCH `/settings/invoice`	প্রিফিক্স, ভাষা, নিচের লেখা।
5	**নেগেটিভ স্টক**	PATCH `/settings/inventory`	allow_negative_stock; চালু করলে audit_logs ও সতর্কতা।



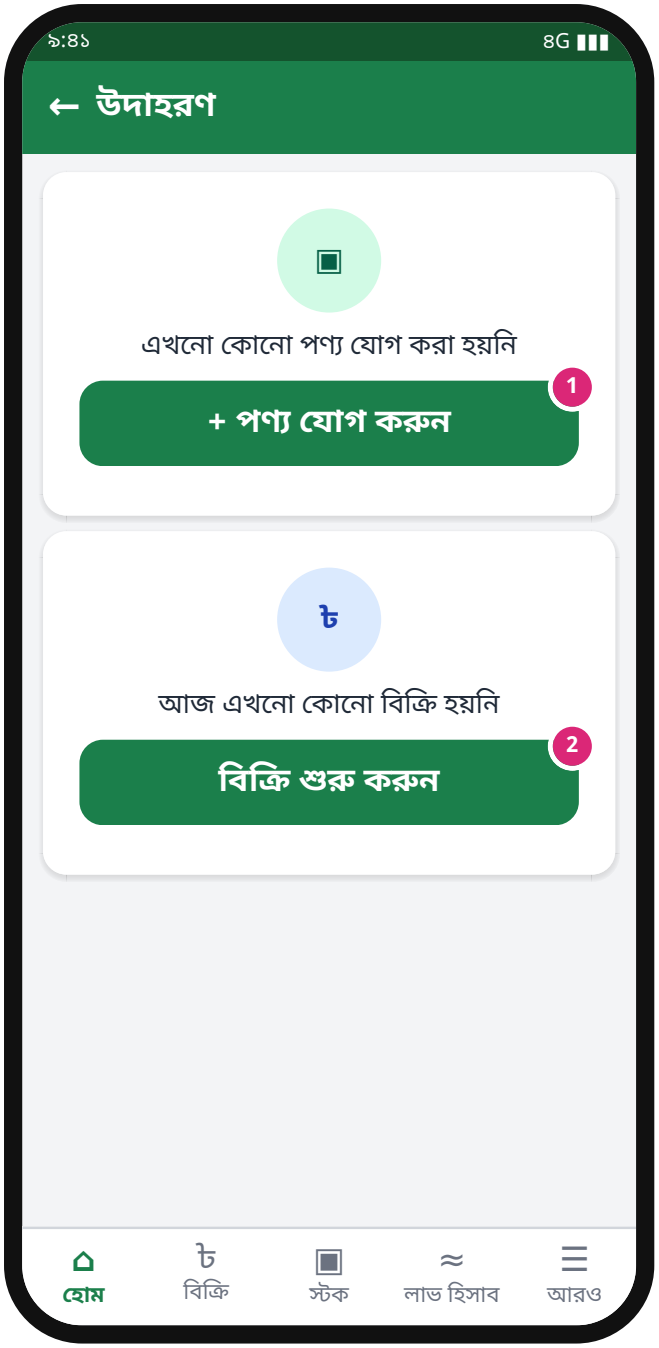
Owner: সব। Staff: কেবল দেওয়া অনুমতি (বিক্রি, ক্রয়, পণ্য/ক্রেতা দেখা)।

#	বাটন / অংশ	API	ব্যাকএন্ডে কী ঘটবে (টেবিল)
1	**মালিক**	GET `/users`	business_members + roles।
2	**স্টাফ সারি**	PATCH `/users/:id`	ভূমিকা/স্ট্যাটাস বদল; নিষ্ক্রিয় করলে সেশন বাতিল।
3	**অনুমতি টগল**	PATCH `/users/:id/permissions`	role_permissions; সার্ভারেই প্রতিটি API-তে চেক। বদল audit_logs-এ।
4	**নতুন কর্মী**	POST `/users/invite`	invitation + অস্থায়ী পাসওয়ার্ড; ডাটা ঐ ব্যবসার মধ্যেই সীমাবদ্ধ।



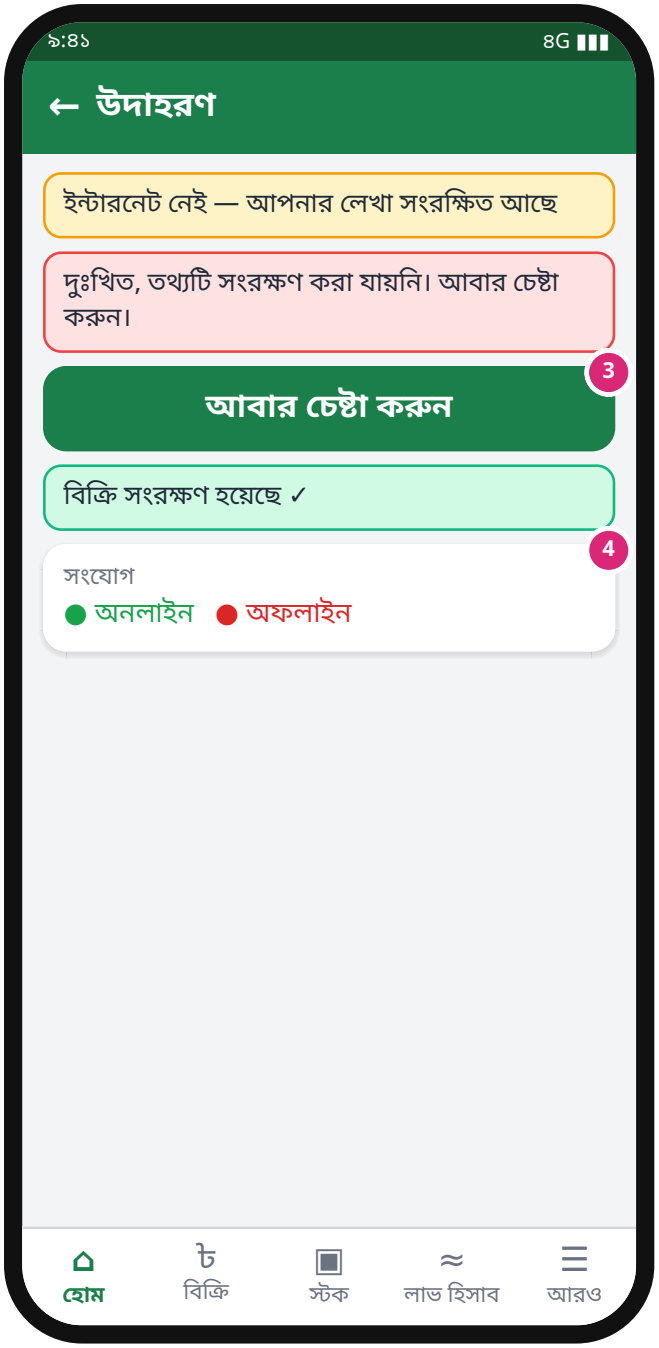
লগ শুধু পড়া যায় — মোছা/বদলানো যায় না।

#	বাটন / অংশ	API	ব্যাকএন্ডে কী ঘটবে (টেবিল)
1	**ফিল্টার**	GET `/audit-logs?action=&from&to`	শুধু Owner।
2	**সারি**	GET `/audit-logs/:id`	user, action, সময়, entity_type/id, আগের/পরের মান, কারণ।
3	**সারি-ট্যাপ**	UI → মূল রেকর্ড	

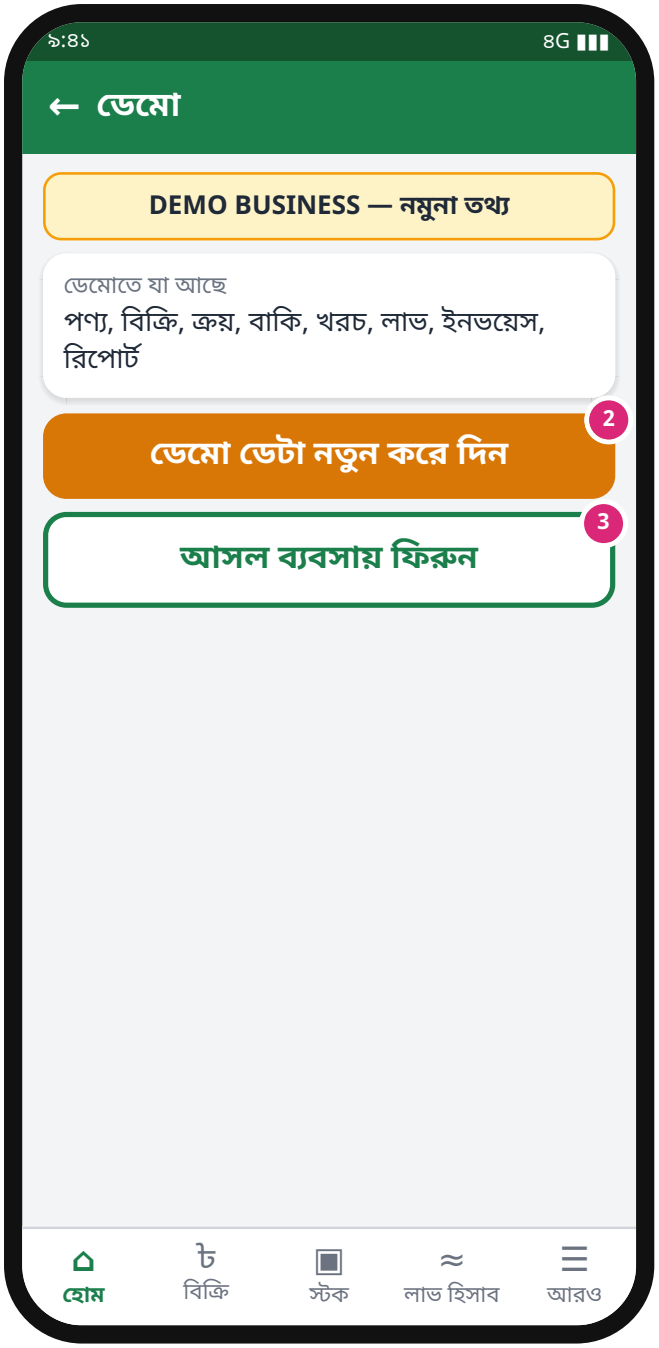


প্রতিটি তালিকায় (ক্রেতা, খরচ, ইনভয়েস, রিপোর্ট) একই ধাঁচে বাংলা বার্তা ও একটি বাটন — কখনো ফাঁকা সাদা পাতা নয়।

#	বাটন / অংশ	API	ব্যাকএন্ডে কী ঘটবে (টেবিল)
1	**পণ্য যোগ করুন**	UI → M07	GET /products খালি এলে এই স্ক্রিন।
2	**বিক্রি শুরু করুন**	UI → M09	GET /sales?today খালি হলে।

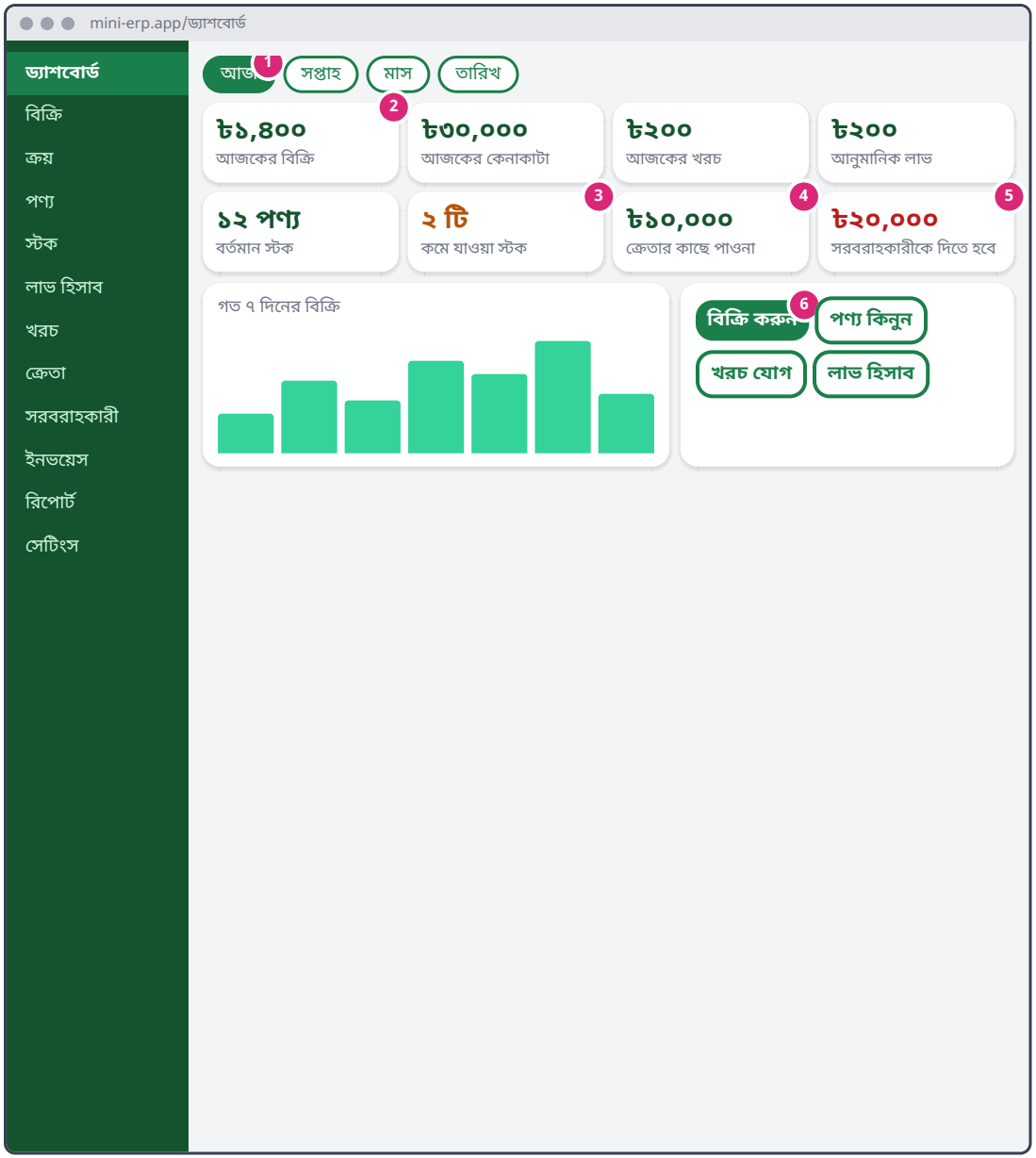


#	বাটন / অংশ	API	ব্যাকএন্ডে কী ঘটবে (টেবিল)
1	**অফলাইন বার**	UI `navigator.onLine`	অসম্পূর্ণ ফর্ম ফোনে সংরক্ষিত (draft)। **পূর্ণ অফলাইন লেনদেন এখন নেই** — Phase 2; এমন দাবি করা হবে না।
2	**ত্রুটি বার্তা**	UI –	সার্ভারের ত্রুটি কোড → বাংলা বার্তা ম্যাপ; টেকনিক্যাল বিবরণ শুধু সার্ভার লগে।
3	**আবার চেষ্টা**	POST একই `Idempotency-Key`	নিরাপদে পুনরায় পাঠানো — ডুপ্লিকেট হয় না।
4	**সংযোগ স্ট্যাটাস**	UI `GET /health`	সবসময় দেখা যায়।



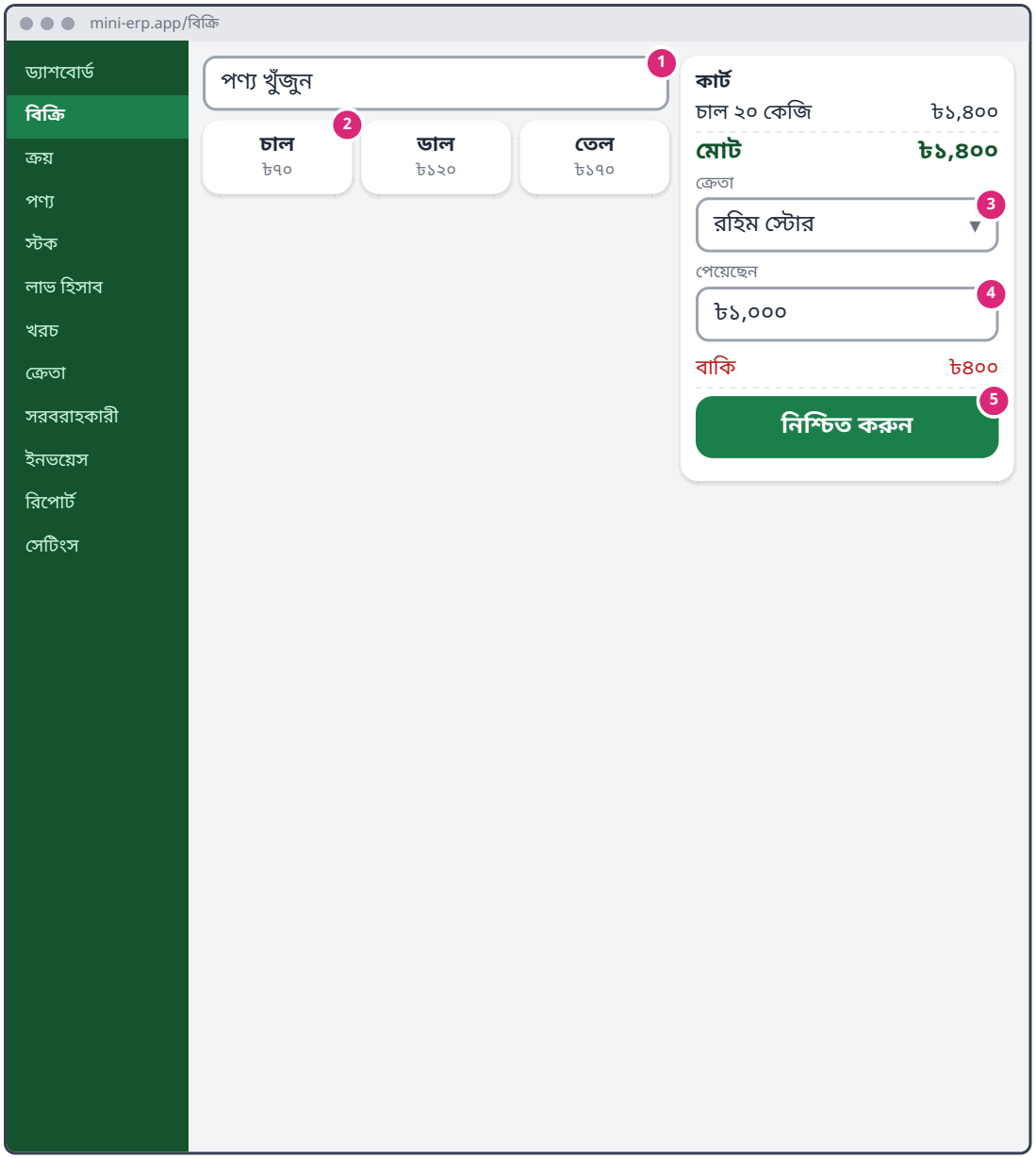
ডেমো ধাপ: ড্যাশবোর্ড → পণ্য (চাল, ৬০/৭০, ১০০ কেজি) → বিক্রি ২০ কেজি (স্টক ৮০, বিক্রি ৳১,৪০০) → বাকি → এস্টিমেটর (৫০০ কেজি) → দাম বদল → ইনভয়েস → রিপোর্ট → একই ব্যবসা মোবাইলে।

#	বাটন / অংশ	API	ব্যাকএন্ডে কী ঘটবে (টেবিল)
1	**DEMO ব্যানার**	UI `business.is_demo = true`	প্রতিটি স্ক্রিনে সবসময় দেখা যায়।
2	**রিসেট**	POST `/demo/reset`	শুধু ডেমো ব্যবসার ডেটা মুছে seed আবার বসায়; আসল ব্যবসায় কাজ করে না।
3	**আসল ব্যবসায় ফিরুন**	POST `/auth/switch-business`	সেশন বদল; ডেটা কখনো মেশে না।



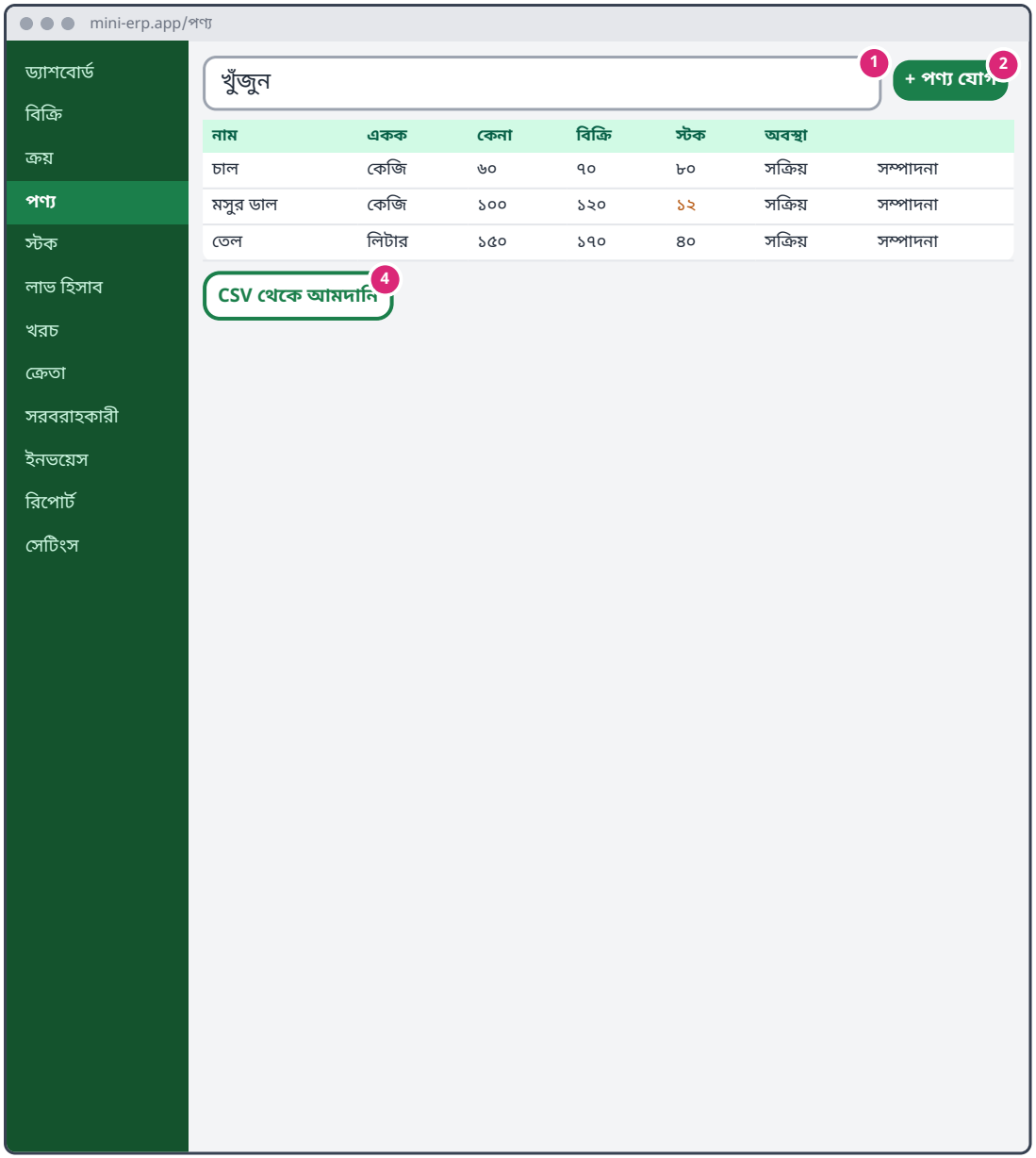
ওয়েব: বাঁ মেনু + সরল চার্ট (সর্বোচ্চ একটি, জটিল চার্ট নয়)। মোবাইল ও ওয়েবের সংখ্যা একই API থেকে।

#	বাটন / অংশ	API	ব্যাকএন্ডে কী ঘটবে (টেবিল)
1	**সময় ফিল্টার**	GET `/reports/dashboard?range=`	মোবাইলের M05-এর একই API।
2	**আজকের বিক্রি**	GET (একই)	SUM(sales.total) active only
3	**কমে যাওয়া স্টক**	GET `/inventory/low-stock`	
4	**ক্রেতার পাওনা**	GET `/payments/receivables/summary`	
5	**সরবরাহকারীর দেনা**	GET `/payments/payables/summary`	
6	**বিক্রি করুন ইত্যাদি**	UI → W02 / W04 / W06	কুইক অ্যাকশন



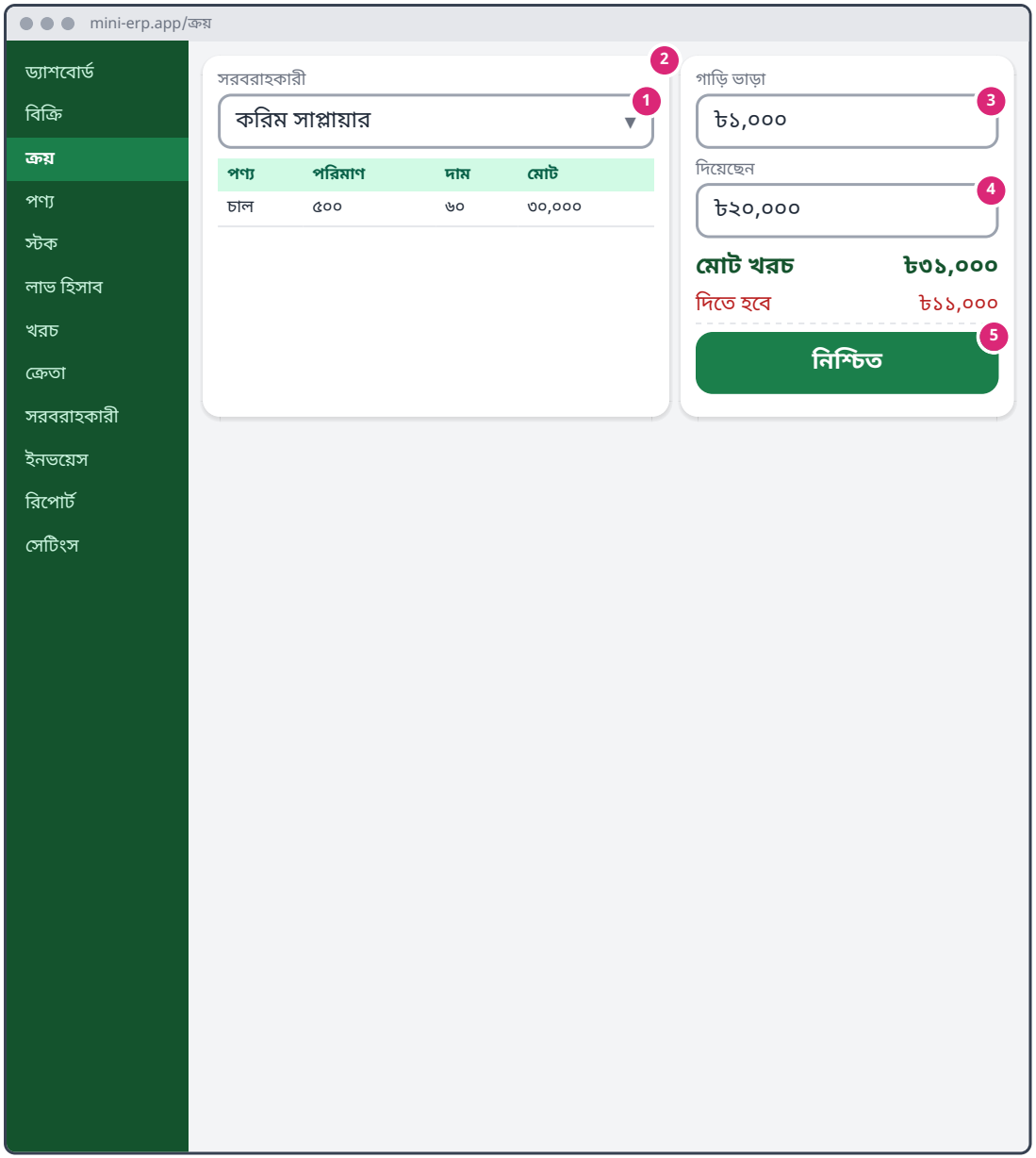
ডেস্কটপে এক স্ক্রিনেই কার্ট ও পণ্য — মোবাইলে ধাপে ধাপে।

#	বাটন / অংশ	API	ব্যাকএন্ডে কী ঘটবে (টেবিল)
1	**খোঁজা**	GET `/products?search=`	
2	**পণ্য কার্ড**	UI –	ক্লিকে কার্টে যোগ; দাম স্বয়ংক্রিয়।
3	**ক্রেতা**	GET `/customers`	
4	**পাওয়া টাকা**	UI –	বাকি অটো।
5	**নিশ্চিত**	POST `/sales (Idempotency-Key)`	M12-এর একই ট্রানজ্যাকশন (sales, sale_items, inventory_movements, payments, customers.balance, invoices, audit_logs)।

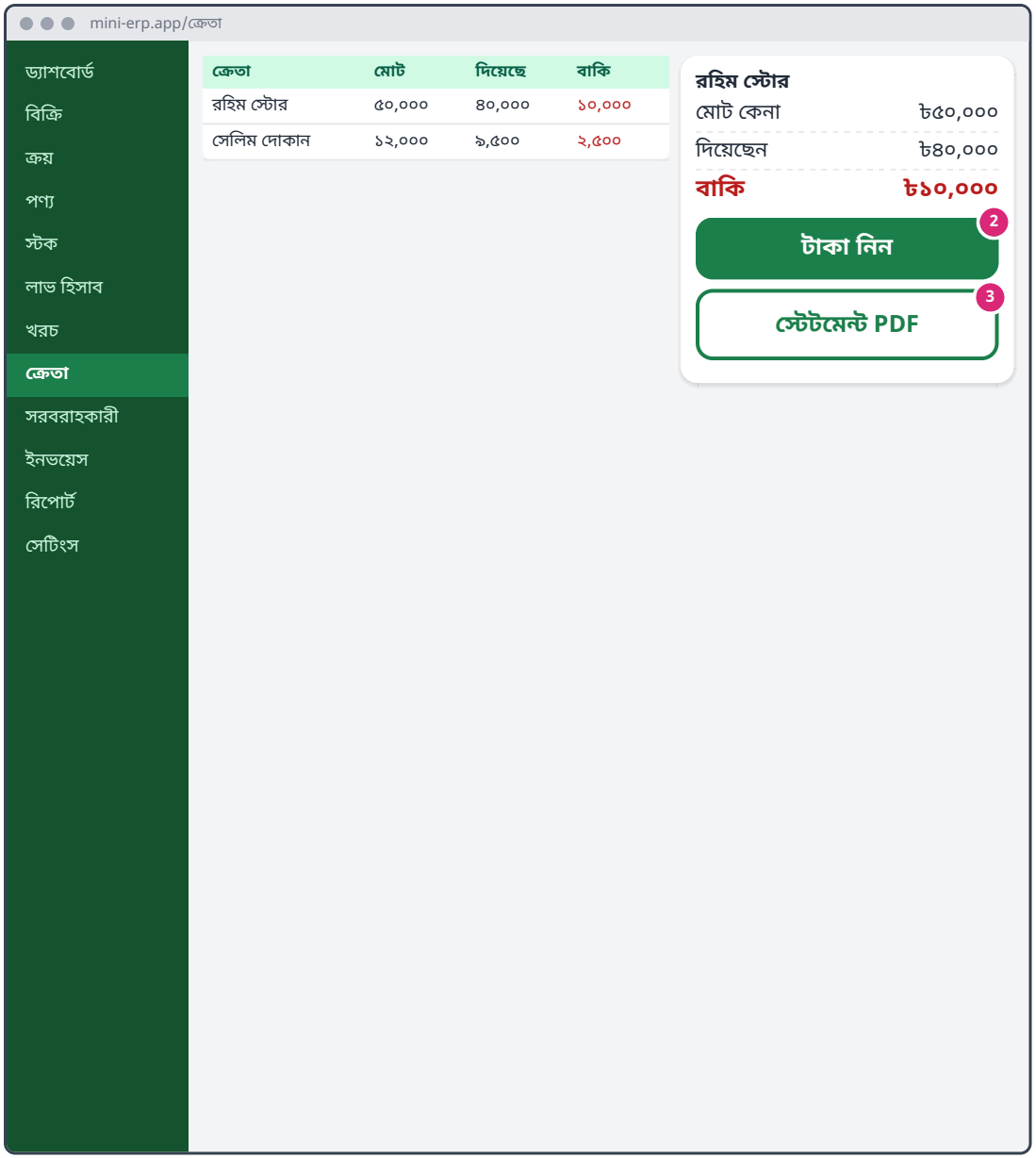


ডেস্কটপে টেবিল ঠিক আছে; মোবাইলে টেবিল নয়, কার্ড।

#	বাটন / অংশ	API	ব্যাকএন্ডে কী ঘটবে (টেবিল)
1	**খোঁজা**	`GET /products?search=`	
2	**+ পণ্য**	`POST /products`	M07-এর মতো।
3	**সম্পাদনা**	`PATCH /products/:id`	দাম বদলে audit_logs।
4	**CSV আমদানি (ঐচ্ছিক)**	`POST /products/import`	ভ্যালিডেশনের পর ব্যাচ ট্রানজ্যাকশন; ত্রুটি সারি বাংলায় দেখায়। MVP-এর পরে হলেও চলে।

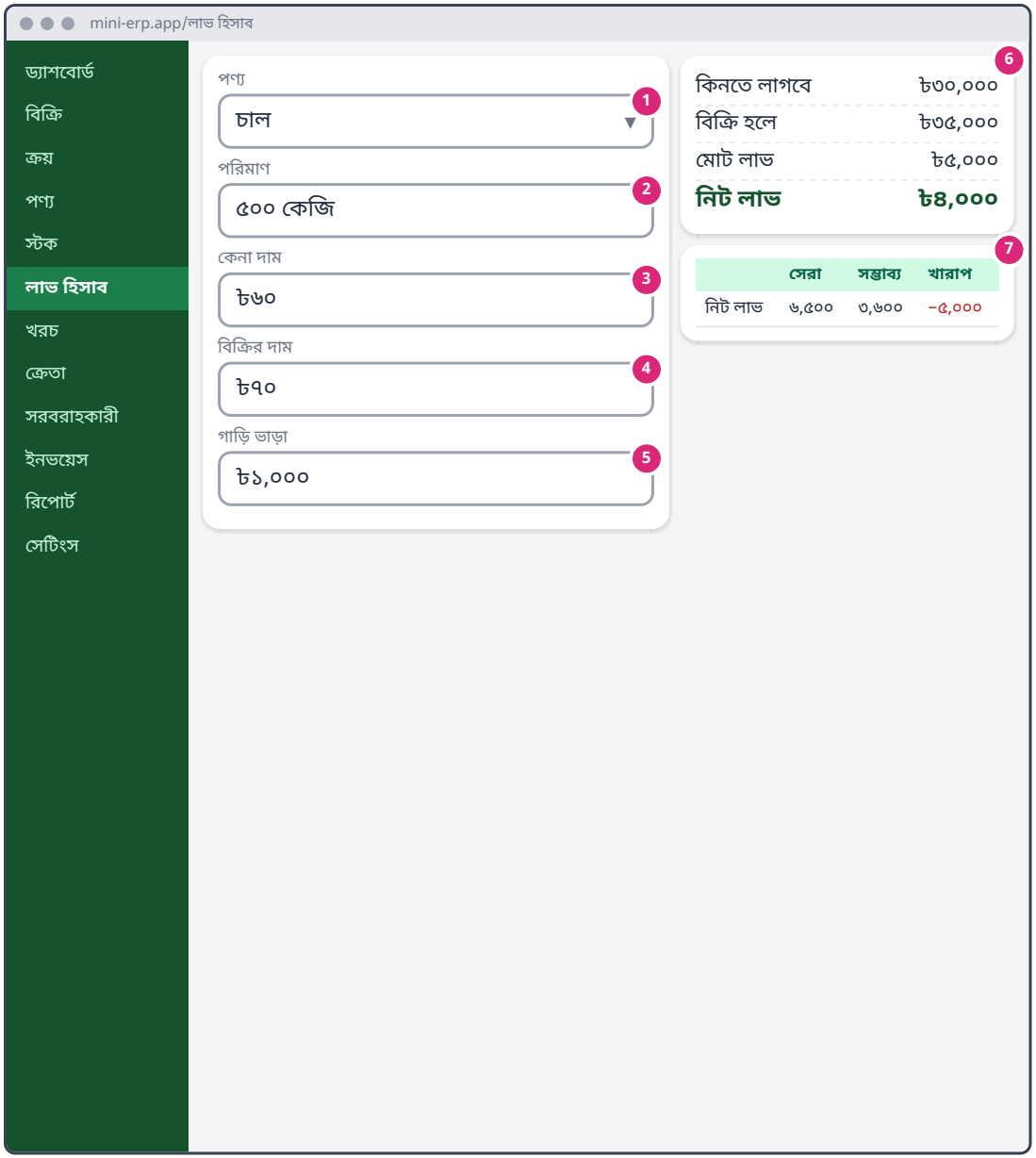


#	বাটন / অংশ	API	ব্যাকএন্ডে কী ঘটবে (টেবিল)
1	**সরবরাহকারী**	GET `/suppliers`	
2	**পণ্য-সারি**	UI –	মোট অটো।
3	**খরচ**	UI –	
4	**দেওয়া টাকা**	UI –	
5	**নিশ্চিত**	POST `/purchases` (Idempotency-Key)	M16-এর একই ট্রানজ্যাকশন।

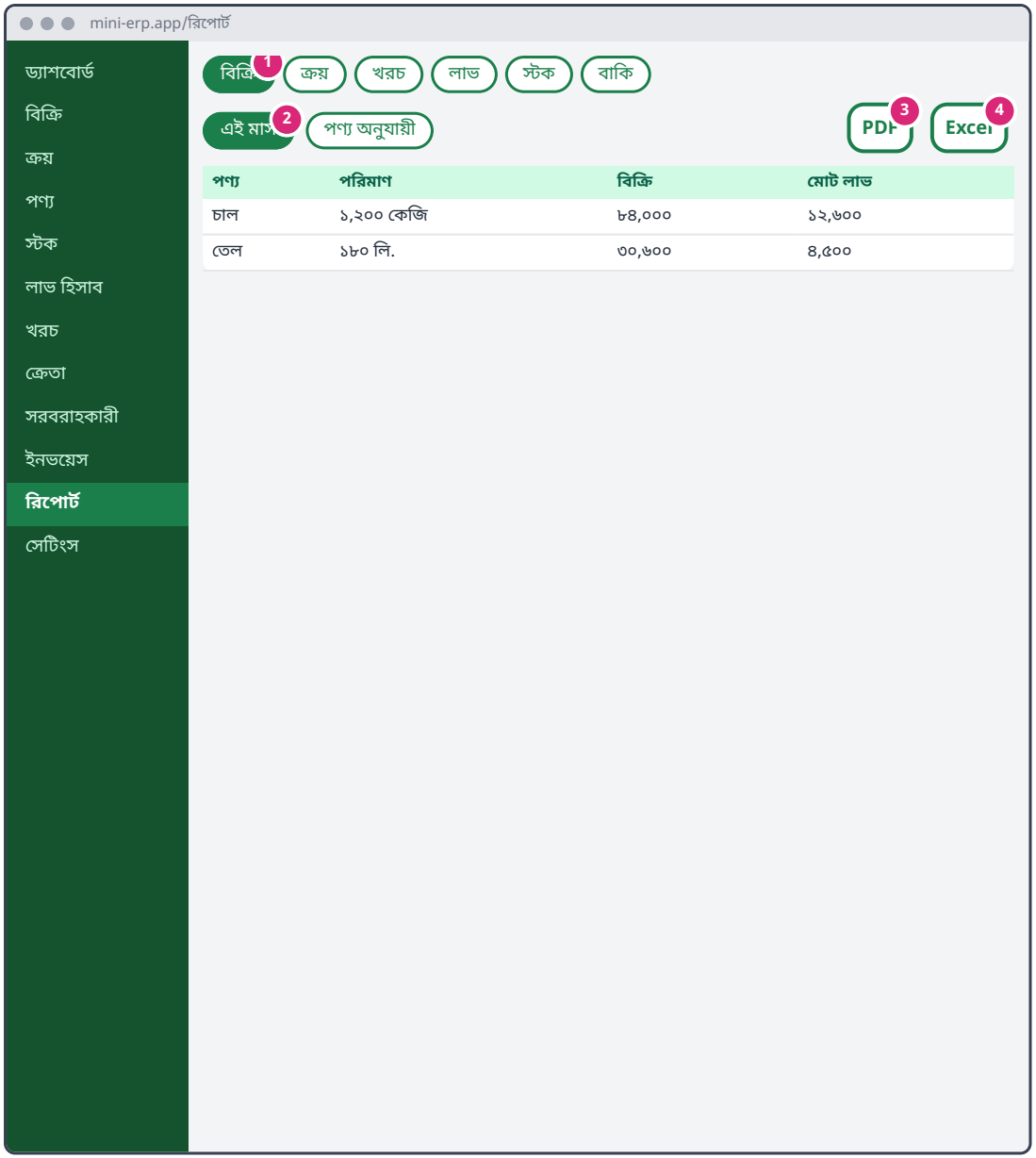


সরবরাহকারীর পাতা একই কাঠামোয় (/suppliers, /payments/pay)।

#	বাটন / অংশ	API	ব্যাকএন্ডে কী ঘটবে (টেবিল)
1	**ক্রেতার তালিকা**	GET `/customers?has_due=`	
2	**টাকা নিন**	POST `/payments/receive`	M30-এর মতো ৪ সংখ্যা, ডুপ্লিকেট সুরক্ষা।
3	**স্টেটমেন্ট**	GET `/customers/:id/statement.pdf`	লেনদেনের ইতিহাসের PDF।

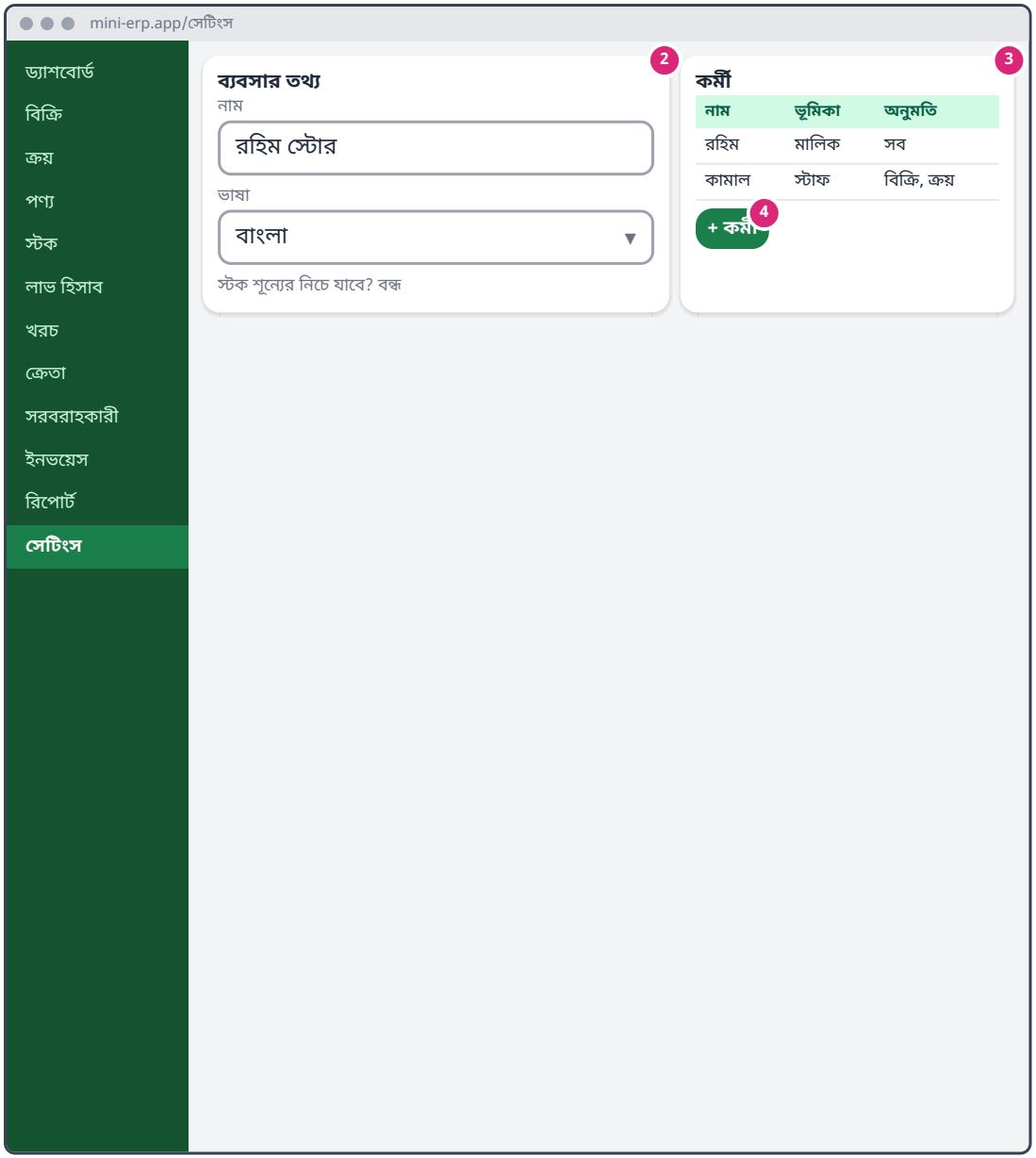


#	বাটন / অংশ	API	ব্যাকএন্ডে কী ঘটবে (টেবিল)
1	**পণ্য**	GET `/products`	
2	**পরিমাণ/দাম/খরচ**	UI –	তাৎক্ষণিক হিসাব, ক্লায়েন্টে (shared/estimator.ts)।
3	**বিক্রির দাম**	UI –	বদলালে লাভ সঙ্গে সঙ্গে বদলায় (ডেমো ধাপ ৬)।
4	**গাড়ি/অন্য**	UI –	
5	**অপচয়/ছাড়/ভলিউম**	UI –	
6	**ফলাফল**	POST `/estimator/calculate`	আসল ডেটা অপরিবর্তিত; সংরক্ষণ করলে estimates টেবিল।
7	**তিন অবস্থা**	POST `/estimator/scenarios`	estimate_scenarios।

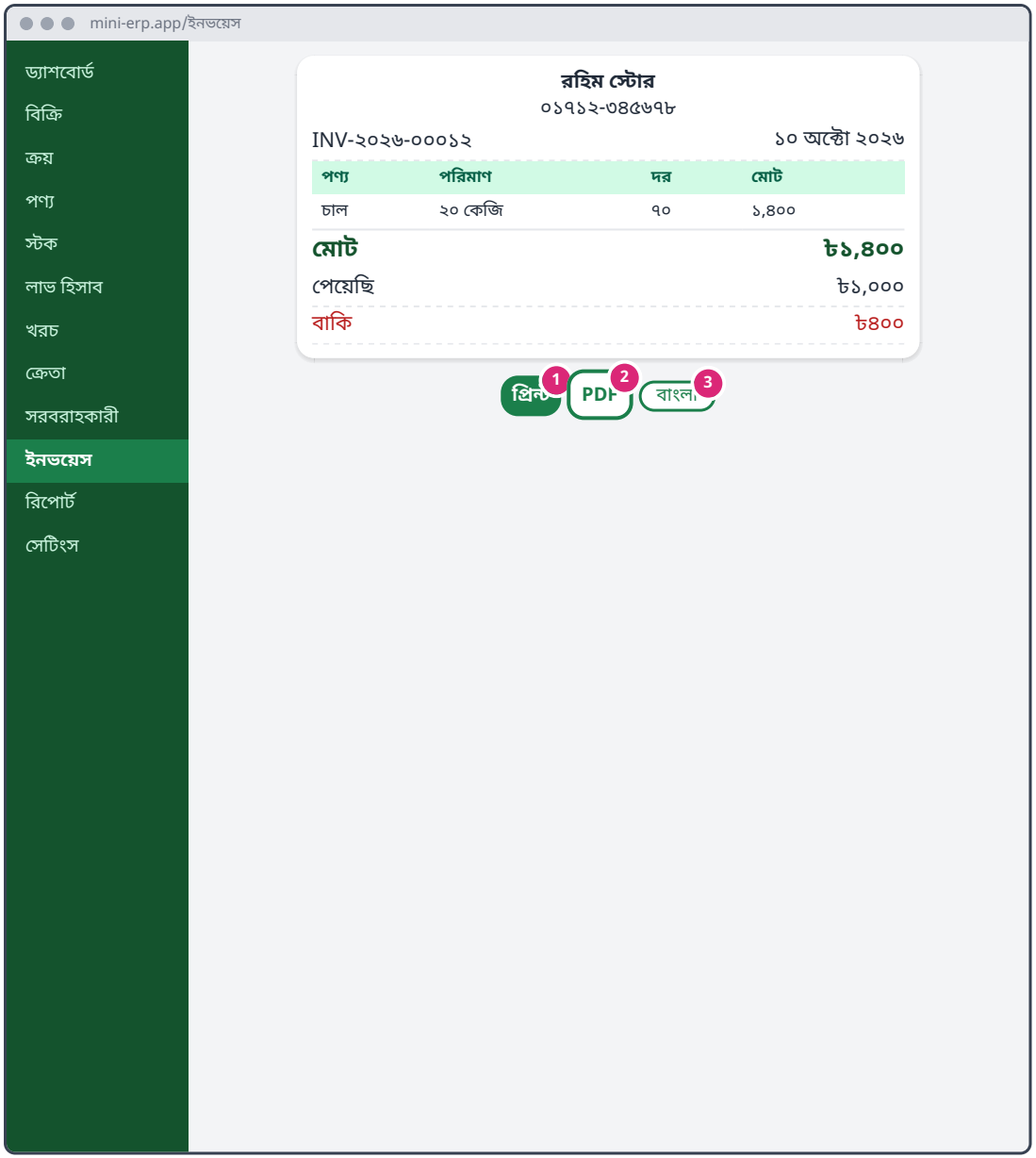


রিপোর্ট সবসময় আসল সংরক্ষিত লেনদেন থেকে; হার্ডকোড নয়।

#	বাটন / অংশ	API	ব্যাকএন্ডে কী ঘটবে (টেবিল)
1	**রিপোর্টের ধরন**	`GET /reports/{sales|purchases|expenses|profit|inventory|dues}`	
2	**সময়/গ্রুপ**	`GET ?period=&group=`	day/week/month/year; product/customer/supplier/category।
3	**PDF**	`GET /reports/:type/pdf`	
4	**Excel/CSV (ঐচ্ছিক)**	`GET /reports/:type/export.csv`	



#	বাটন / অংশ	API	ব্যাকএন্ডে কী ঘটবে (টেবিল)
1	ব্যবসার তথ্য	PATCH /settings/business	
2	নেগেটিভ স্টক	PATCH /settings/inventory	
3	অনুমতি	PATCH /users/:id/permissions	সার্ভারে চেক; audit_logs।
4	নতুন কর্মী	POST /users/invite	



পুরনো ইনভয়েস লেনদেনের দামেই থাকে।

#	বাটন / অংশ	API	ব্যাকএন্ডে কী ঘটবে (টেবিল)
1	**প্রিন্ট**	POST `/invoices/:id/print`	ব্রাউজার প্রিন্ট; reprint_count++।
2	**PDF**	GET `/invoices/:id/pdf`	
3	**ভাষা**	GET `/invoices/:id?lang=`	

Mini ERP — UI ও ব্যাকএন্ড ম্যাপ

ছোট ব্যবসার জন্য সহজ বাংলা ব্যবসা-ব্যবস্থাপনা (মোবাইল Android-first + রেসপন্সিভ ওয়েব)

প্রতিটি স্ক্রিনের লাল নম্বরযুক্ত বাটনের পাশে লেখা: কোন API, কোন HTTP মেথড, কোন ডাটাবেস টেবিল ও কী ব্যবসায়িক নিয়ম।

৪৫টি মোবাইল স্ক্রিন (M01–M45) · ৯টি ওয়েব স্ক্রিন (W01–W09) · API তালিকা · ডাটাবেস ও ট্রানজ্যাকশন নিয়ম · ডেমো ধারা

এগুলো ডিজাইন মকআপ — আসল ফ্রন্টএন্ড কোড নয়। ব্যাকএন্ড নির্মাণের নির্দেশিকা হিসেবে তৈরি।

পাতার ধরন

বাঁ পাশে ফোন/ওয়েব স্ক্রিন, ডানে নম্বর-ধরে টেবিল: **বাটন → API → ব্যাকএন্ডে কী ঘটবে**।

মেথডের রং

GET পড়া POST তৈরি PATCH বদল DELETE আর্কাইভ UI সার্ভার কল নেই

সব লেনদেন-API-র সাধারণ নিয়ম

- সব রিকোয়েস্টে টোকেন; ব্যবসার আইডি টোকেন থেকে (ক্লায়েন্টের পাঠানো আইডি বিশ্বাস নয়)।
- টাকা পাঠানো POST-এ `Idempotency-Key` হেডার — ডুপ্লিকেট রোধ।
- সার্ভারে ভ্যালিডেশন (shared/validation), ত্রুটি বাংলা বার্তায়।
- টাকার কলাম NUMERIC(14,2); কোনো আর্থিক রেকর্ড ডিলিট নয়, উল্টো এন্ট্রি।

মোবাইল স্ক্রিন সূচি

M01–M04	ভাষা, লগইন, সাইন আপ, পাসওয়ার্ড রিসেট
M05	হোম ড্যাশবোর্ড
M06–M08	পণ্য তালিকা, যোগ, বিস্তারিত
M09–M14	বিক্রি (বাছা, কার্ট, টাকা, যাচাই, সফল, ইতিহাস/বাতিল)
M15–M17	ক্রয় ও ফেরত
M18–M20	স্টক, ইতিহাস, কম স্টক সতর্কতা
M21–M22	খরচ
M23–M25	লাভ হিসাব, What-if, তিন অবস্থা
M26	আরও মেনু
M27–M33	ক্রেতা, সরবরাহকারী, বাকি-পাওনা, পেমেন্ট
M34–M35	ইনভয়েস
M36–M39	রিপোর্ট
M40–M42	সেটিংস, ব্যবহারকারী, অডিট লগ
M43–M45	ফাঁকা অবস্থা, ত্রুটি/অফলাইন, ডেমো

ওয়েব

W01 ড্যাশবোর্ড · W02 বিক্রি · W03 পণ্য · W04 ক্রয় · W05 ক্রেতা · W06 লাভ হিসাব · W07 রিপোর্ট · W08 সেটিংস · W09 ইনভয়েস

একটি ট্রানজ্যাকশনের ভেতরে (সব হবে, নয়তো কিছুই না)

ধাপ	টেবিল	কী ঘটে
1	`SELECT … FOR UPDATE (products)`	স্টক লক করে যাচাই; নেগেটিভ হলে বাতিল
2	`sales`	হেডার: মোট, ছাড়, পাওনা, স্ট্যাটাস=completed
3	`sale_items`	পণ্য, পরিমাণ, তখনকার বিক্রি দাম ও কেনা দাম (snapshot)
4	`inventory_movements`	type=sale, −পরিমাণ, source_id=sale_id
5	`products.current_stock`	কমে (movement থেকে মিলিয়ে)
6	`payments`	পাওয়া টাকা (in)
7	`customers.balance`	+ (মোট − পাওয়া)
8	`invoices`	নম্বর অটো; তখনকার তথ্য
9	`audit_logs`	sale_created
10	`idempotency_keys`	কী + ফলাফল সংরক্ষণ

ফেল করলে

যেকোনো ধাপ ব্যর্থ হলে পুরো ট্রানজ্যাকশন ROLLBACK। ব্যবহারকারী দেখে: “দুঃখিত, তথ্যটি সংরক্ষণ করা যায়নি। আবার চেষ্টা করুন।” আর্থিক রেকর্ডের অর্ধেক সেভ হয় না।

ডুপ্লিকেট রোধ

বাটন একবার চাপলে নিষ্ক্রিয়; একই Idempotency-Key আবার এলে আগের ফলাফলই ফেরত।

বাতিল / সংশোধন

ডিলিট নয় — রিভার্সাল movement ও উল্টো payment; কারণ বাধ্যতামূলক; audit_logs-এ আগের/পরের মান।

হিসাব সূত্র

Revenue = বিক্রির টাকা
Gross Profit = Revenue − COGS (sale_items-এর কেনা দাম × পরিমাণ)
Net Profit = Gross Profit − খরচ

একটি ট্রানজ্যাকশনের ভেতরে (সব হবে, নয়তো কিছুই না)

ধাপ	টেবিল	কী ঘটে
1	`purchases`	মোট, গাড়ি ভাড়া, অন্য খরচ
2	`purchase_items`	পণ্য, পরিমাণ, কেনা দাম
3	`inventory_movements`	type=purchase, +পরিমাণ
4	`products.current_stock`	বাড়ে
5	`payments`	দেওয়া টাকা (out)
6	`suppliers.balance`	+ (মোট − দেওয়া)
7	`audit_logs`	purchase_created
8	`idempotency_keys`	

ফেল করলে

যেকোনো ধাপ ব্যর্থ হলে পুরো ট্রানজ্যাকশন ROLLBACK। ব্যবহারকারী দেখে: “দুঃখিত, তথ্যটি সংরক্ষণ করা যায়নি। আবার চেষ্টা করুন।” আর্থিক রেকর্ডের অর্ধেক সেভ হয় না।

ডুপ্লিকেট রোধ

বাটন একবার চাপলে নিষ্ক্রিয়; একই Idempotency-Key আবার এলে আগের ফলাফলই ফেরত।

বাতিল / সংশোধন

ডিলিট নয় — রিভার্সাল movement ও উল্টো payment; কারণ বাধ্যতামূলক; audit_logs-এ আগের/পরের মান।

হিসাব সূত্র

Revenue = বিক্রির টাকা
Gross Profit = Revenue − COGS (sale_items-এর কেনা দাম × পরিমাণ)
Net Profit = Gross Profit − খরচ

API সম্পূর্ণ API তালিকা (সারাংশ)

Express + TypeScript

গ্রুপ	এন্ডপয়েন্ট
/auth	POST register, login, logout, password-reset/request|confirm, demo-login, switch-business
/products	GET list|:id · POST · PATCH :id · PATCH :id/price · PATCH :id/status · DELETE :id (archive) · POST import
/categories	GET · POST · PATCH :id
/units	GET · POST
/sales	POST preview · POST (create) · GET list|:id · PATCH :id/correct · POST :id/cancel
/purchases	POST preview · POST · GET list|:id · POST :id/return · POST :id/cancel · PATCH :id/correct
/inventory	GET list · GET low-stock · GET :productId/movements · GET :id/suggestion · POST adjustments · POST :id/snooze
/customers	GET list|:id · GET :id/summary|transactions|statement.pdf · POST · PATCH :id
/suppliers	GET list|:id · GET :id/summary|purchases · POST · PATCH :id
/payments	GET list · GET receivables|payables/summary · POST receive|pay (+ /preview)
/expenses	GET list · GET categories · POST · PATCH :id · DELETE :id
/invoices	GET list|:id|by-sale/:saleId · GET :id/pdf · POST :id/print
/reports	GET dashboard · sales · purchases · expenses · profit · inventory · dues (+ /pdf, export.csv)
/estimator	POST calculate · POST scenarios · GET scenarios · GET scenarios/:id/pdf
/users	GET · POST invite · PATCH :id · PATCH :id/permissions
/settings	GET · PATCH business|language|invoice|inventory
/audit-logs	GET list|:id (শুধু Owner)
/demo	POST reset (শুধু ডেমো ব্যবসা)
/health	GET

রেসপন্সের একই ছাঁচ

```
{ "ok": true, "data": {…}, "message": "বাংলা বার্তা" }
{ "ok": false, "error": { "code": "STOCK_LOW", "message": "স্টকে এত নেই" } }
```

নিরাপত্তা

- প্রতিটি কোয়েরিতে `business_id = token.business_id`।
- Role/Permission মিডলওয়্যার (Owner/Staff)।
- পাসওয়ার্ড argon2/bcrypt; গোপন কী .env-এ।
- রেট লিমিট, ইনপুট ভ্যালিডেশন (zod), HTTPS, CORS সীমিত।
- প্রতিটি ফাইন্যান্সিয়াল ক্রিয়ায় audit_logs।

ত্রুটি কোড

VALIDATION_ERROR, UNAUTHORIZED, FORBIDDEN, NOT_FOUND, STOCK_LOW, DUPLICATE_REQUEST, INTERNAL — সবগুলো বাংলা বার্তায় রূপান্তর।

DB ডাটাবেস (PostgreSQL) — টেবিল সারাংশ

NUMERIC(14,2) · FK · CHECK constraint

users
id, name, phone(unique), password_hash

businesses
id, name, type, is_demo

business_members
business_id, user_id, role_id

roles / role_permissions
Owner, Staff; অনুমতি

products
business_id, name_bn, unit, purchase_price, selling_price, current_stock, min_stock, is_active, archived_at

categories
business_id, name

customers
business_id, name, phone, balance

suppliers
business_id, name, phone, balance

sales
business_id, customer_id, total, discount, paid, due, status, idempotency_key

sale_items
sale_id, product_id, qty, unit_price, cost_price_snapshot, line_total

purchases
business_id, supplier_id, total, transport_cost, other_cost, paid, status

purchase_items
purchase_id, product_id, qty, unit_cost

inventory_movements
product_id, type, qty(+/−), source_type, source_id, reason, user_id, created_at

expenses
business_id, category, amount, date, method, note

payments
business_id, direction(in/out), amount, method, party_id, ref_type, ref_id

invoices
sale_id, number, lang, printed_count, snapshot

audit_logs
business_id, user_id, action, entity, entity_id, before, after, reason, at

settings
business_id, language, allow_negative_stock, ...

estimates / estimate_scenarios
আলাদা — আসল ডেটার সাথে সম্পর্কহীন

idempotency_keys
key, user_id, response, created_at

password_resets
user_id, code_hash, expires_at

গুরুত্বপূর্ণ নিয়ম

• প্রতিটি টেবিলে business_id (ইনডেক্সসহ) • টাকা = NUMERIC, কখনো float নয় • current_stock ≥ 0 CHECK (সেটিং অনুযায়ী) • পণ্য+একক ইউনিক ব্যবসাভিত্তিক • কোনো আর্থিক সারি DELETE নয় • সব বদলে created_by/updated_by • migrations ও seeds আলাদা ফোল্ডারে।

ডেমো — ৯ ধাপ

১	ড্যাশবোর্ড খুলুন (M05/W01)
২	পণ্য যোগ: চাল, ৬০/৭০, ১০০ কেজি (M07)
৩	২০ কেজি বিক্রি → স্টক ৮০, বিক্রি ৳১,৪০০ (M09–M13)
৪	ক্রেতার বাকি দেখান (M28)
৫	এস্টিমেটর: ৫০০ কেজি কিনলে? (M23)
৬	বিক্রির দাম বদলে লাভ বদলান (M24)
৭	ইনভয়েস তৈরি (M35)
৮	রিপোর্টে লেনদেন (M37–M38)
৯	একই ব্যবসা মোবাইলে (একই API/ডেটাবেস)

বাস্তবায়নের ক্রম (আপনার ব্যাকএন্ডের জন্য)

Phase 1	ডাটাবেস, auth, ব্যবসা/ব্যবহারকারী, settings (M01–M04, M40–M41)
Phase 2	products, categories, customers, suppliers, sales, purchases, inventory (M06–M20, M27–M29, M31)
Phase 3	payments, dues, expenses, profit (M21–M22, M30, M32–M33)
Phase 4	estimator, low-stock পরামর্শ (M20, M23–M25)
Phase 5	invoices, reports (M34–M39)
Phase 6–8	মোবাইল, কোয়ালিটি, ডেমো (M43–M45)

মূল পরীক্ষা (বাধ্যতামূলক)

বিক্রিতে স্টক কমে · আংশিক টাকায় বাকি ঠিক · ক্রয়ে স্টক বাড়ে · স্টক ট্রেসযোগ্য · কারণ ছাড়া অ্যাডজাস্ট নয় · এস্টিমেটর আসল ডেটা ছোঁয় না · ইনভয়েসে পুরনো দাম · অন্য ব্যবসার ডেটা দেখা যায় না · স্টাফ অনুমতি মানা হয় · ডুপ্লিকেট রিকোয়েস্টে ডুপ্লিকেট লেনদেন নয়।

সীমাবদ্ধতা: এগুলো মকআপ; পূর্ণ অফলাইন লেনদেন, ভয়েস ইনপুট ও নোটিফিকেশন Phase 2-এ — MVP-তে দাবি করা হয়নি।